Frontend Angular/TypeScript Interview Handbook

Gated Funnel Structure for .NET Full-Stack Developer Interviews

Use this handbook as both **interviewer** (evaluate candidates) and **candidate** (self-assessment). Questions are organized into 4 gates by experience level. A candidate must demonstrate competence at their gate before advancing.

Gate	Experience	Question Style	Pass Threshold
Gate 1	Junior (0-3 yr)	Direct conceptual	70% correct
Gate 2	Mid (3-7 yr)	Direct + scenario-based	65% correct, can reason through scenarios
Gate 3	Senior (7-15 yr)	Mostly scenario-based	Can propose tradeoffs, not just answers
Gate 4	Lead/Architect (15+ yr)	All design/scenario-based	Demonstrates system-level thinking

Table of Contents

PART A — JavaScript Fundamentals

- Q1: What is a closure and why are closures useful?
- Q2: What will this code output and why?
- Q3: Classic closure pitfall — the loop problem
- Q4: How does JavaScript inheritance work under the hood?
- Q5: Predict the output — event loop ordering
- Q6: Scenario — Diagnosing a frozen UI
- Q7: Convert callback to Promise/async-await
- Q8: Scenario — Parallel vs sequential async operations
- Q9: Destructuring, spread, and rest — what does this output?
- Q10: Predict the output — `this` binding
- Q11: When would you use event delegation, and how does it work?

PART B — TypeScript

- Q12: When would you use an interface vs a type alias?
- Q13: Build a type-safe API response handler using generics and utility types
- Q14: Scenario — Handling polymorphic API responses
- Q15: Explain the differences between `any`, `unknown`, and `never`
- Q16: Scenario — How do Angular decorators work, and what happens at runtime?

PART C — Angular Deep Dive

- Q17: What is the difference between NgModules and Standalone Components?
- Q18: Explain `providedIn` and hierarchical injectors
- Q19: Set up lazy loading with route guards
- Q20: Scenario — Route resolver vs guard vs component initialization
- Q21: Explain the difference between `switchMap`, `mergeMap`, `concatMap`, and `exhaustMap`
- Q22: Scenario — Memory leaks with RxJS in Angular
- Q23: Design a reactive state management solution without NgRx
- Q24: Template-driven vs Reactive Forms — when to use which?
- Q25: Scenario — Performance debugging with change detection
- Q26: List the Angular lifecycle hooks in order and explain when to use the 3 most important ones
- Q27: Implement an HTTP interceptor for auth tokens and error handling
- Q28: Scenario — API error handling strategy
- Q29: Scenario — Choosing a state management approach
- Q30: Scenario — Bundle size and load time optimization
- Q31: What are the most important Angular CLI commands you use daily?
- Q32: What are Angular Signals and how do they compare to RxJS Observables?
- Q33: Scenario — Deep component tree communication
- Q34: Write a unit test for an Angular service that calls an API
- Q35: Scenario — Testing a component with complex dependencies
- Q36: Design a micro-frontend architecture with Angular
- Q37: Design a real-time collaborative feature using Angular
- Q38: Scenario — Securing an Angular application

PART D — HTML/CSS Essentials

- Q39: When do you use Flexbox vs CSS Grid?
- Q40: Implement a responsive layout strategy
- Q41: How does CSS specificity work?
- Q42: What are the key accessibility practices for web applications?

Appendices

- Appendix A — Angular CLI Quick Reference
- Appendix B — RxJS Operators Cheat Sheet
- Gate Assessment Summary

PART A — JavaScript Fundamentals

A1. Closures, Scope Chain, and Hoisting

Gate 1 | Q1: What is a closure and why are closures useful?

Question: Explain what a closure is in JavaScript. Give a practical example.

Expected Answer:

A closure is a function that retains access to its lexical scope even after the outer function has returned. The inner function "closes over" the variables of the outer function.

```
function createCounter() {
  let count = 0; // enclosed variable
  return {
    increment: () => ++count,
    getCount: () => count,
  };
}

const counter = createCounter();
counter.increment();
counter.increment();
console.log(counter.getCount()); // 2
// `count` is not accessible directly, but the returned methods still reference it
```

Practical uses: data privacy/encapsulation, factory functions, memoization, maintaining state in callbacks.

What to listen for:

- Uses the phrase "lexical scope" or "outer scope" naturally
- Mentions that the variable survives past the outer function's return
- Can name a real use case beyond the textbook example

Gate 1 | Q2: What will this code output and why?

Question:

```
console.log(a); // ?
console.log(b); // ?
var a = 1;
let b = 2;
```

Expected Answer:

- `console.log(a)` prints `undefined` because `var` declarations are hoisted to the top of the function scope, but the assignment stays in place. So `a` exists but has not been assigned yet.
- `console.log(b)` throws a `ReferenceError` because `let` declarations are hoisted but are in the **Temporal Dead Zone (TDZ)** until the declaration line is reached. Accessing them before that is an error.

What to listen for:

- Mentions hoisting for both `var` and `let`
- Correctly identifies TDZ for `let`/`const`
- Does not say "`let` is not hoisted" (it is hoisted, but the TDZ prevents access)

Gate 2 | Q3: Classic closure pitfall — the loop problem

Question: What does this code print, and how would you fix it?

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
```

Expected Answer:

It prints `3, 3, 3` because `var i` is function-scoped (or global-scoped). By the time the `setTimeout` callbacks fire, the loop has completed and `i` is `3`.

Fixes:

```
// Fix 1: Use let (block-scoped — each iteration gets its own `i`)
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// prints 0, 1, 2

// Fix 2: IIFE to capture the value
for (var i = 0; i < 3; i++) {
  ((j) => {
    setTimeout(() => console.log(j), 100);
  })(i);
}

// Fix 3: setTimeout's third argument
for (var i = 0; i < 3; i++) {
  setTimeout((j) => console.log(j), 100, i);
}
```

What to listen for:

- Correctly identifies `var` scoping as the root cause
- Knows the `let` fix and at least one other approach
- Can explain *why* each fix works, not just that it works

Red flag: Says "setTimeout is broken" or cannot explain why `var` behaves differently from `let` in loops.

A2. Prototypes and Prototype Chain

Gate 2 | Q4: How does JavaScript inheritance work under the hood?

Question: Explain the prototype chain. How does property lookup work when you access `obj.property`?

Expected Answer:

Every JavaScript object has an internal `[[Prototype]]` link (accessible via `__proto__` or `Object.getPrototypeOf()`). When you access a property on an object:

1. The engine checks the object itself.
2. If not found, it follows `[[Prototype]]` to the parent object.
3. This continues up the chain until the property is found or `null` is reached (end of chain).

```
const animal = {
  speak() { return '...'; }
};

const dog = Object.create(animal);
dog.bark = function() { return 'Woof!'; };

dog.bark();   // found on dog itself
dog.speak();  // not on dog -> found on animal (prototype)
dog.toString(); // not on dog -> not on animal -> found on Object.prototype
```

`class` syntax is syntactic sugar over this prototype mechanism:

```
class Animal {
  speak() { return '...'; }
}

class Dog extends Animal {
  bark() { return 'Woof!'; }
}

// Dog.prototype.__proto__ === Animal.prototype
```

What to listen for:

- Describes the chain as a linked list of objects
- Knows that `class` is sugar over prototypes, not a separate system
- Can explain `Object.create()` and how it sets up the chain

Red flag: Believes JavaScript has "real" classical inheritance or cannot explain what happens when a property is not found on the object itself.

A3. Event Loop, Microtasks vs Macrotasks

Gate 2 | Q5: Predict the output — event loop ordering

Question: What is the output order and why?

```
console.log('1');

setTimeout(() => console.log('2'), 0);

Promise.resolve().then(() => console.log('3'));

queueMicrotask(() => console.log('4'));

console.log('5');
```

Expected Answer:

Output: `1, 5, 3, 4, 2`

Explanation:

1. `1` and `5` run synchronously (call stack).
2. After the synchronous code completes, the **microtask queue** is drained: `3` (Promise `.then`) and `4` (`queueMicrotask`).
3. Then the **macrotask queue** runs: `2` (`setTimeout`).

The event loop processes all microtasks before moving to the next macrotask. Microtasks include Promise callbacks, `queueMicrotask`, and `MutationObserver`. Macrotasks include `setTimeout`, `setInterval`, `setImmediate` (Node), and I/O callbacks.

What to listen for:

- Correct output order
- Uses the terms "microtask" and "macrotask" (or "task queue")
- Knows that microtasks drain completely before the next macrotask

Red flag: Thinks `setTimeout(fn, 0)` runs immediately or before Promise callbacks.

Gate 3 | Q6: Scenario — Diagnosing a frozen UI

Question: A developer reports that the browser freezes for 2-3 seconds after the user clicks "Generate Report." The click handler calls an API, processes the response into a table, then renders it. The API responds in 200ms. What is your debugging approach, and what event-loop-related issue might be causing this?

Expected Answer:

The freeze indicates the **main thread is blocked** during the synchronous processing/rendering phase (not the API call, which is async). Likely causes:

1. **Heavy synchronous computation** after the API response: e.g., sorting/filtering/transforming thousands of rows on the main thread. Solution: break work into chunks using `requestIdleCallback`, `setTimeout` batching, or move to a **Web Worker**.

2. **Excessive DOM manipulation**: inserting thousands of DOM nodes synchronously triggers layout thrashing. Solution: use `DocumentFragment`, virtual scrolling, or batch rendering with

`requestAnimationFrame`.

3. **Accidental synchronous operation**: a `.forEach` that inadvertently blocks. Check with the Performance tab in DevTools — look for a long task on the main thread.

Debugging steps:

- Open DevTools Performance tab, record the click-to-render flow
- Look for long tasks (>50ms) in the flame chart
- Check if the bottleneck is JS execution, layout, or paint
- Use `performance.mark()` and `performance.measure()` to instrument the code

What to listen for:

- Immediately suspects synchronous processing, not the network call
- Mentions Web Workers or chunked processing as solutions
- Knows how to use DevTools Performance profiler
- Considers DOM thrashing as a separate concern from JS computation

Red flag: Suggests "make the API faster" without investigating the client-side processing. Cannot explain why async API calls would not cause freezing.

A4. Promises, async/await, Error Handling

Gate 1 | Q7: Convert callback to Promise/async-await

Question: Rewrite this callback-based function using async/await with proper error handling:

```
function loadUser(userId, callback) {
  fetch(`/api/users/${userId}`)
    .then(res => res.json())
    .then(data => callback(null, data))
    .catch(err => callback(err, null));
}
```

Expected Answer:

```
async function loadUser(userId) {
  const response = await fetch(`/api/users/${userId}`);

  if (!response.ok) {
    throw new Error(`Failed to load user: ${response.status}
${response.statusText}`);
  }

  return response.json();
}

// Usage
```

```
try {
  const user = await loadUser(42);
  console.log(user);
} catch (error) {
  console.error('Could not load user:', error.message);
}
```

What to listen for:

- Checks `response.ok` (fetch does not reject on HTTP errors like 404/500)
- Uses try/catch for error handling
- Returns the parsed data instead of using a callback
- Does not forget to `await` the `response.json()` call

Gate 3 | Q8: Scenario — Parallel vs sequential async operations

Question: You need to load a user's profile, their recent orders, and their notification settings before rendering a dashboard. These three API calls are independent. A junior developer wrote this:

```
async function loadDashboard(userId) {
  const profile = await fetchProfile(userId);
  const orders = await fetchOrders(userId);
  const notifications = await fetchNotifications(userId);
  return { profile, orders, notifications };
}
```

What is wrong and how would you fix it? What if one of them failing should not prevent the others from loading?

Expected Answer:

The calls are sequential (each awaits before the next starts). Since they are independent, they should run in parallel:

```
// Fix 1: Promise.all — fail-fast if any rejects
async function loadDashboard(userId) {
  const [profile, orders, notifications] = await Promise.all([
    fetchProfile(userId),
    fetchOrders(userId),
    fetchNotifications(userId),
  ]);
  return { profile, orders, notifications };
}

// Fix 2: Promise.allSettled — resilient, partial data is acceptable
async function loadDashboard(userId) {
  const results = await Promise.allSettled([
    fetchProfile(userId),
```

```
    fetchOrders(userId),
    fetchNotifications(userId),
  ]);

  return {
    profile: results[0].status === 'fulfilled' ? results[0].value : null,
    orders: results[1].status === 'fulfilled' ? results[1].value : null,
    notifications: results[2].status === 'fulfilled' ? results[2].value : null,
  };
}
```

If one failing should not block others, `Promise.allSettled` is the right tool. `Promise.all` short-circuits on the first rejection.

What to listen for:

- Immediately identifies the sequential bottleneck
- Knows the difference between `Promise.all` and `Promise.allSettled`
- Considers the partial-failure scenario without being prompted
- Can estimate the time savings (sequential: sum of all latencies; parallel: max of all latencies)

Red flag: Does not know `Promise.allSettled` exists. Suggests wrapping each call in try/catch inside `Promise.all` without mentioning `allSettled`.

A5. ES6+ Features

Gate 1 | Q9: Destructuring, spread, and rest — what does this output?

Question: Explain what each line does:

```
const user = { name: 'Alex', age: 30, role: 'dev', team: 'frontend' };

const { name, ...rest } = user;

const updated = { ...rest, role: 'lead', level: 'senior' };

console.log(name);    // ?
console.log(rest);    // ?
console.log(updated); // ?
```

Expected Answer:

```
console.log(name);    // 'Alex'
console.log(rest);    // { age: 30, role: 'dev', team: 'frontend' }
console.log(updated); // { age: 30, role: 'lead', team: 'frontend', level:
'senior' }
```

- `{ name, ...rest } = user` destructures `name` out and collects the remaining properties into `rest` (rest syntax).
- `{ ...rest, role: 'lead', level: 'senior' }` spreads `rest` into a new object, then overrides `role` (because it appears after the spread) and adds `level`.

Key point: property order matters with spread. Later properties overwrite earlier ones with the same key.

What to listen for:

- Correctly predicts all three outputs
- Explains that spread creates a shallow copy
- Knows that later properties override earlier ones

A6. `this` Keyword and Arrow Functions

Gate 2 | Q10: Predict the output — `this` binding

Question: What does each `console.log` print?

```
const obj = {
  name: 'Widget',
  getName: function() {
    return this.name;
  },
  getNameArrow: () => {
    return this.name;
  },
  getNameDelayed: function() {
    setTimeout(function() {
      console.log('A:', this.name);
    }, 0);
    setTimeout(() => {
      console.log('B:', this.name);
    }, 0);
  },
};

console.log(obj.getName());       // ?
console.log(obj.getNameArrow()); // ?
obj.getNameDelayed();             // A: ? B: ?
```

Expected Answer:

```
'Widget'
undefined  (or '' depending on environment)
A: undefined  (or '' — `this` is the global object / window)
B: 'Widget'
```

Explanation:

- `getName()` — regular function called as a method, `this` is `obj`.
- `getNameArrow()` — arrow function does not have its own `this`; it captures `this` from the enclosing lexical scope (the module/global scope), not from `obj`.
- `getNameDelayed()`:
  - Callback A: regular function passed to `setTimeout`. When invoked, `this` is the global object (not `obj`), so `this.name` is `undefined`.
  - Callback B: arrow function captures `this` from `getNameDelayed`, which was called as `obj.getNameDelayed()`, so `this` is `obj` and `this.name` is `'Widget'`.

What to listen for:

- Understands that arrow functions capture `this` lexically, not at call time
- Knows that regular functions in `setTimeout` lose their `this` binding
- Can explain the fix: arrow function, `.bind(this)`, or `const self = this`

Red flag: Believes arrow functions always point `this` to the containing object literal.

A7. Event Delegation, Bubbling, and Capturing

Gate 2 | Q11: When would you use event delegation, and how does it work?

Question: You have a list of 1,000 items. Each item needs a click handler. How would you implement this efficiently?

Expected Answer:

Use **event delegation**: attach a single event listener to the parent element instead of 1,000 individual listeners. This works because of **event bubbling** — events fired on a child propagate up through the DOM tree.

```
// Instead of this (1000 listeners):
document.querySelectorAll('.item').forEach(item => {
  item.addEventListener('click', handleClick);
});

// Do this (1 listener):
document.querySelector('.item-list').addEventListener('click', (event) => {
  const item = event.target.closest('.item');
  if (!item) return; // click was not on an item

  const itemId = item.dataset.id;
  handleItemClick(itemId);
});
```

Event propagation phases:

1. **Capturing** — event travels from `window` down to the target (rarely used)

2. **Target** — event reaches the clicked element
3. **Bubbling** — event travels back up from target to `window` (default)

`event.stopPropagation()` prevents further propagation. `event.preventDefault()` prevents the default browser action (e.g., form submission, link navigation).

What to listen for:

- Uses `event.target.closest()` for robust delegation (handles nested elements inside the item)
- Mentions memory and performance benefits
- Understands all three phases
- Knows the difference between `stopPropagation` and `preventDefault`

Red flag: Attaches individual listeners and says "it's fine for 1,000 items."

↑ Back to Index

PART B — TypeScript

B1. Types, Interfaces, and Type Aliases

Gate 1 | Q12: When would you use an interface vs a type alias?

Question: What is the difference between `interface` and `type` in TypeScript? When would you choose one over the other?

Expected Answer:

```
// Interface — best for object shapes, especially public APIs
interface User {
  id: number;
  name: string;
  email: string;
}

// Interfaces can be extended
interface AdminUser extends User {
  permissions: string[];
}

// Interfaces support declaration merging
interface User {
  avatar?: string; // this merges with the original User interface
}

// Type alias — more flexible, works for unions, intersections, primitives
type Status = 'active' | 'inactive' | 'banned';
type ApiResponse<T> = { data: T; error: null } | { data: null; error: string };
type Coordinate = [number, number];
```

Guidelines:

- **Interface** when defining object shapes, class contracts, or library APIs (declaration merging is useful for consumers).
- **Type alias** when you need unions, intersections, mapped types, conditional types, tuples, or primitives.
- In Angular projects, the convention is often to use `interface` for DTOs/models and `type` for everything else.

What to listen for:

- Mentions declaration merging as a distinguishing feature
- Knows that types can do things interfaces cannot (unions, conditional types)
- Does not say "they are exactly the same" or "always use one over the other"

B2. Generics and Utility Types

Gate 2 | Q13: Build a type-safe API response handler using generics and utility types

Question: Create a generic type for API responses that supports success and error states. Then show how you would use `Partial`, `Pick`, and `Omit` in a practical context.

Expected Answer:

```
// Generic API response
interface ApiSuccess<T> {
  status: 'success';
  data: T;
  timestamp: number;
}

interface ApiError {
  status: 'error';
  message: string;
  code: number;
  timestamp: number;
}

type ApiResponse<T> = ApiSuccess<T> | ApiError;

// Usage
async function fetchUser(id: number): Promise<ApiResponse<User>> {
  try {
    const response = await fetch(`/api/users/${id}`);
    const data = await response.json();
    return { status: 'success', data, timestamp: Date.now() };
  } catch (e) {
    return { status: 'error', message: (e as Error).message, code: 500, timestamp:
Date.now() };
  }
}
```

```
// Consuming with type narrowing
const result = await fetchUser(1);
if (result.status === 'success') {
  console.log(result.data.name); // TypeScript knows `data` is User here
}

// Utility types in practice
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
  createdAt: Date;
}

type UserUpdatePayload = Partial<Pick<User, 'name' | 'email'>>;
// { name?: string; email?: string }

type PublicUser = Omit<User, 'password'>;
// { id: number; name: string; email: string; createdAt: Date }

type UserLookup = Record<number, PublicUser>;
// { [key: number]: PublicUser }
```

What to listen for:

- Uses discriminated union for success/error (not optional fields)
- Demonstrates type narrowing with the discriminant (`status`)
- Combines utility types (e.g., `Partial<Pick<...>>`)
- Can explain what each utility type does and when to use it

Red flag: Uses `any` for the response data type. Cannot explain what `Partial` does without looking it up.

B3. Type Guards and Discriminated Unions

Gate 3 | Q14: Scenario — Handling polymorphic API responses

Question: Your API returns different shapes based on the entity type. A junior developer uses `as` type assertions everywhere. How would you redesign this for type safety?

```
// Junior's approach:
const data = await fetchEntity(id);
if (entityType === 'user') {
  const user = data as User;
  // ...
}
```

Expected Answer:

Use **discriminated unions** with **type guards** to eliminate `as` assertions:

```
// Define discriminated union
interface UserEntity {
  kind: 'user';
  id: number;
  name: string;
  email: string;
}

interface OrderEntity {
  kind: 'order';
  id: number;
  total: number;
  items: OrderItem[];
}

interface ProductEntity {
  kind: 'product';
  id: number;
  sku: string;
  price: number;
}

type Entity = UserEntity | OrderEntity | ProductEntity;

// Type guard function
function isUserEntity(entity: Entity): entity is UserEntity {
  return entity.kind === 'user';
}

// Usage — TypeScript narrows automatically
function processEntity(entity: Entity) {
  switch (entity.kind) {
    case 'user':
      // TypeScript knows entity is UserEntity here
      console.log(entity.email);
      break;
    case 'order':
      // TypeScript knows entity is OrderEntity here
      console.log(entity.total);
      break;
    case 'product':
      console.log(entity.sku);
      break;
    default:
      // Exhaustiveness check — if a new kind is added, this will error at compile
time
      const _exhaustive: never = entity;
      throw new Error(`Unhandled entity kind: ${(_exhaustive as any).kind}`);
  }
}
```

The `never` exhaustiveness check ensures that if a new entity type is added to the union, the developer gets a compile-time error until they handle it.

What to listen for:

- Uses a discriminant property (`kind`, `type`, `tag`) consistently
- Demonstrates the `never` exhaustiveness check pattern
- Explains why `as` assertions are dangerous (they bypass the type checker)
- Mentions custom type guard functions (`entity is UserEntity`)

Red flag: Proposes keeping `as` assertions but wrapping them in utility functions. Does not know about the `never` exhaustiveness trick.

B4. `any` vs `unknown` vs `never`

Gate 2 | Q15: Explain the differences between `any`, `unknown`, and `never`

Question: When would you use each of these types? Give a practical example for each.

Expected Answer:

```
// any — opts out of type checking entirely. Avoid unless absolutely necessary.
// Use case: migrating JS to TS incrementally, third-party library with no types
let legacy: any = fetchFromLegacySystem();
legacy.anything.goes.here; // no error, no safety

// unknown — the type-safe counterpart of any.
// You must narrow it before using it.
function processInput(input: unknown) {
  // input.toUpperCase(); // ERROR: Object is of type 'unknown'

  if (typeof input === 'string') {
    input.toUpperCase(); // OK — narrowed to string
  }

  if (input instanceof Error) {
    input.message; // OK — narrowed to Error
  }
}

// never — represents values that never occur.
// Use case 1: Functions that never return
function throwError(message: string): never {
  throw new Error(message);
}

// Use case 2: Exhaustiveness checking
type Shape = 'circle' | 'square';
function getArea(shape: Shape) {
  switch (shape) {
```

```
    case 'circle': return Math.PI;
    case 'square': return 1;
    default:
      const _: never = shape; // compile error if a new shape is added but not
handled
      return _;
  }
}
```

Summary:

Type	Assignment	Usage without narrowing	Purpose
`any`	Anything assignable	Anything callable	Escape hatch (avoid)
`unknown`	Anything assignable	Nothing — must narrow first	Safe "I don't know yet"
`never`	Nothing assignable	N/A	Impossibility / exhaustiveness

What to listen for:

- Correctly states that `unknown` requires narrowing while `any` does not
- Gives the exhaustiveness check as a use case for `never`
- Advises against `any` in new code
- Knows that `unknown` is assignable *to* nothing (without narrowing) but anything is assignable *to* `unknown`

B5. Decorators in Angular Context

Gate 3 | Q16: Scenario — How do Angular decorators work, and what happens at runtime?

Question: A junior developer asks: "How does `@Component` actually turn a class into an Angular component? Is it magic?" Explain what happens.

Expected Answer:

Decorators are functions that receive the decorated target and can modify it or attach metadata. In Angular's case, decorators attach metadata that the Angular compiler and runtime use to configure the framework.

```
// Simplified conceptual model of what @Component does:
function Component(config: ComponentConfig) {
  return function(target: Function) {
    // Attach metadata to the class
    Reflect.defineMetadata('annotations', [config], target);
    // Angular's compiler reads this metadata to:
    // 1. Associate the template/styles with this class
    // 2. Register it in the dependency injection system
    // 3. Set up change detection for it
    // 4. Create a component factory
  };
}
```

```
// When you write:
@Component({
  selector: 'app-dashboard',
  templateUrl: './dashboard.component.html',
  styleUrls: ['./dashboard.component.scss'],
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class DashboardComponent { }

// It is equivalent to:
export class DashboardComponent { }
Component({
  selector: 'app-dashboard',
  templateUrl: './dashboard.component.html',
  styleUrls: ['./dashboard.component.scss'],
  changeDetection: ChangeDetectionStrategy.OnPush,
})(DashboardComponent);
```

Common Angular decorators and what they do:

- `@Component` — class is a component (template + logic)
- `@Injectable` — class can be injected via DI
- `@NgModule` — class is a module (declarations, imports, providers)
- `@Input()` / `@Output()` — property decorators for component I/O
- `@ViewChild` / `@ContentChild` — query decorators for DOM/component references
- `@HostListener` / `@HostBinding` — bind to host element events/properties

With Angular's Ivy compiler (Angular 9+), much of this metadata is compiled ahead of time into static fields on the class (`ɵcmp`, `ɵfac`, `ɵinj`) rather than relying on `Reflect.defineMetadata` at runtime.

What to listen for:

- Explains decorators as functions, not "magic"
- Mentions metadata attachment
- Knows about Ivy and AOT compilation changing how decorators are processed
- Can explain at least 3-4 Angular decorators beyond `@Component`

Red flag: Cannot explain what a decorator is at the JavaScript level. Thinks decorators are an Angular invention rather than a TypeScript/JavaScript feature.

↑ Back to Index

PART C — Angular Deep Dive

C1. Components, Modules, and Standalone Components

Gate 1 | Q17: What is the difference between NgModules and Standalone Components?

Question: Angular 14+ introduced standalone components. Explain the difference and when you might still use NgModules.

Expected Answer:

NgModules (traditional):

```
// feature.module.ts
@NgModule({
  declarations: [FeatureComponent, FeaturePipe],
  imports: [CommonModule, SharedModule],
  exports: [FeatureComponent],
})
export class FeatureModule { }

// feature.component.ts
@Component({
  selector: 'app-feature',
  templateUrl: './feature.component.html',
})
export class FeatureComponent { }
```

Standalone Components (Angular 14+):

```
@Component({
  selector: 'app-feature',
  standalone: true,
  imports: [CommonModule, SharedComponent, FeaturePipe],
  templateUrl: './feature.component.html',
})
export class FeatureComponent { }
```

Key differences:

- Standalone components declare their own dependencies via `imports` — no module wrapper needed.
- Reduces boilerplate (no NgModule file for simple features).
- Simplifies lazy loading (can lazy-load a single component, not a whole module).
- Tree-shaking is more effective because dependencies are explicit per component.

When NgModules are still useful:

- Legacy codebases that have not migrated
- Grouping related providers with `forRoot()`/`forChild()` patterns (e.g., `RouterModule.forRoot()`)
- Libraries that export a bundle of related components/directives/pipes

What to listen for:

- Understands that standalone is the modern direction (Angular 17+ defaults to standalone)
- Mentions tree-shaking and lazy loading benefits

- Does not say "NgModules are deprecated" (they are not, they are just optional)

C2. Services and Dependency Injection

Gate 2 | Q18: Explain `providedIn` and hierarchical injectors

Question: What does `providedIn: 'root'` do? What happens if you provide the same service in a component's `providers` array?

Expected Answer:

```
// Singleton — provided at root level, one instance for the entire app
@Injectable({ providedIn: 'root' })
export class UserService {
  private currentUser: User | null = null;
  // ...
}

// If you also provide it in a component:
@Component({
  selector: 'app-admin',
  providers: [UserService], // creates a NEW instance for this component and its
children
  templateUrl: './admin.component.html',
})
export class AdminComponent {
  constructor(private userService: UserService) {
    // This is a DIFFERENT instance than the root one
  }
}
```

Angular's DI uses a **hierarchical injector tree**:

1. **Root injector** — `providedIn: 'root'` or `AppModule` providers
2. **Module injectors** — lazy-loaded module providers (creates a child injector)
3. **Component injectors** — component `providers` array (creates a child injector per component instance)

Resolution order: Angular looks for the provider starting at the component injector and walks up the tree until it finds one.

`providedIn: 'root'` benefits:

- Tree-shakable: if no component injects it, it is removed from the bundle
- Guaranteed singleton without needing to import a module
- Recommended for most application services

What to listen for:

- Understands hierarchical injector tree and resolution order

- Knows that component-level providers create separate instances
- Mentions tree-shaking as a benefit of `providedIn: 'root'`
- Can explain when you would intentionally provide at a lower level (e.g., per-component state)

Red flag: Thinks `providedIn: 'root'` and providing in `AppModule` are exactly the same (they are not — tree-shaking differs). Does not understand that component-level providers create new instances.

C3. Routing, Guards, Resolvers, and Lazy Loading

Gate 2 | Q19: Set up lazy loading with route guards

Question: Show how to configure a lazy-loaded route with an authentication guard in Angular 15+.

Expected Answer:

```
// app.routes.ts (standalone approach, Angular 15+)
export const routes: Routes = [
  {
    path: 'dashboard',
    canActivate: [authGuard],
    loadComponent: () =>
      import('./features/dashboard/dashboard.component')
        .then(m => m.DashboardComponent),
  },
  {
    path: 'admin',
    canActivate: [authGuard, roleGuard],
    loadChildren: () =>
      import('./features/admin/admin.routes')
        .then(m => m.ADMIN_ROUTES),
  },
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' },
  { path: '**', loadComponent: () =>
      import('./shared/not-found.component').then(m => m.NotFoundComponent) },
];

// auth.guard.ts (functional guard — Angular 15+ preferred style)
export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isAuthenticated()) {
    return true;
  }

  return router.createUrlTree(['/login'], {
    queryParams: { returnUrl: state.url },
  });
};

// role.guard.ts
```

```
export const roleGuard: CanActivateFn = (route) => {
  const authService = inject(AuthService);
  const requiredRole = route.data['role'] as string;
  return authService.hasRole(requiredRole);
};
```

Key concepts:

- `loadComponent` — lazy loads a single standalone component
- `loadChildren` — lazy loads a set of child routes
- `canActivate` — runs before the route activates; can return `boolean`, `UrlTree`, or `Observable<boolean>`
- Functional guards (using `inject()`) are the modern replacement for class-based guards
- `UrlTree` return for redirects (preferred over `router.navigate` inside guards)

What to listen for:

- Uses `loadComponent`/`loadChildren` with dynamic `import()` syntax
- Uses functional guard style with `inject()`
- Returns `UrlTree` for redirects, not calling `router.navigate` + returning false
- Stores returnUrl for post-login redirect

Red flag: Uses class-based guards with `implements CanActivate` (outdated but not wrong). Does not know about `loadComponent` for standalone lazy loading.

Gate 3 | Q20: Scenario — Route resolver vs guard vs component initialization

Question: A page needs user data before it renders. A developer puts the data fetch in `ngOnInit`. Another suggests a route resolver. A third suggests a guard. What are the tradeoffs?

Expected Answer:

Approach	Pros	Cons
`ngOnInit` fetch	Simple, component is independent	Page renders with loading state, skeleton needed; duplicate fetch logic if multiple routes need same data
Route Resolver	Data available before component renders, no loading state needed	Navigation blocks until data loads (bad UX if slow); resolver errors need careful handling
Route Guard	Can prevent navigation if data indicates access denied	Not designed for data fetching — guards are for access control, not data loading

Modern recommendation (Angular 16+):

Use a **resolver** for critical data that must exist before render, but make it **observable-based** so the router can start rendering partial data:

```
// user.resolver.ts
export const userResolver: ResolveFn<User> = (route) => {
  const userService = inject(UserService);
  const userId = Number(route.paramMap.get('id'));

  return userService.getUser(userId).pipe(
    catchError(() => {
      const router = inject(Router);
      router.navigate(['/not-found']);
      return EMPTY;
    }),
  );
};

// Route config
{
  path: 'users/:id',
  loadComponent: () => import('./user-profile.component').then(m =>
m.UserProfileComponent),
  resolve: { user: userResolver },
}

// Component
export class UserProfileComponent {
  private route = inject(ActivatedRoute);
  user = this.route.data.pipe(map(data => data['user']));
}
```

In practice, many teams prefer `ngOnInit` with a loading state (skeleton UI) because it gives the user faster perceived performance — the page shell appears immediately while data loads. Resolvers are better for short-latency data where a loading state would flicker.

What to listen for:

- Discusses tradeoffs, not just "one right answer"
- Mentions UX implications (blocking navigation vs showing loading state)
- Knows that resolvers delay navigation
- Suggests the approach depends on latency and UX requirements

Red flag: Says "always use resolvers" or "always fetch in ngOnInit" without considering tradeoffs.

C4. RxJS

Gate 2 | Q21: Explain the difference between `switchMap`, `mergeMap`, `concatMap`, and `exhaustMap`

Question: When would you use each of these flattening operators?

Expected Answer:

All four operators map each source emission to an inner Observable and flatten the result. They differ in how they handle concurrent inner subscriptions:

```
// switchMap — cancels the previous inner Observable when a new value arrives
// Use for: search typeahead, route parameter changes
this.searchInput$.pipe(
  debounceTime(300),
  switchMap(query => this.searchService.search(query)),
  // If user types "ang" then "angular", the "ang" request is cancelled
);

// mergeMap — runs all inner Observables concurrently
// Use for: fire-and-forget operations where order doesn't matter
this.notificationIds$.pipe(
  mergeMap(id => this.markAsRead(id)),
  // All markAsRead calls run in parallel
);

// concatMap — queues inner Observables, processes one at a time in order
// Use for: sequential operations where order matters
this.fileUploads$.pipe(
  concatMap(file => this.uploadService.upload(file)),
  // Files upload one after another, in order
);

// exhaustMap — ignores new values while an inner Observable is still running
// Use for: preventing duplicate submissions (e.g., button clicks)
this.submitClick$.pipe(
  exhaustMap(() => this.formService.submit(this.form.value)),
  // Rapid clicks are ignored while the first submit is in progress
);
```

Operator	Concurrent inner subs	Cancels previous	Order preserved
`switchMap`	1 (cancels prev)	Yes	N/A (only latest)
`mergeMap`	Unlimited	No	No guarantee
`concatMap`	1 (queues)	No	Yes
`exhaustMap`	1 (drops new)	No	N/A (ignores new)

What to listen for:

- Gives a real use case for each, not just the definition
- Knows `switchMap` cancels (critical for search/autocomplete)
- Mentions `exhaustMap` for duplicate submission prevention
- Can explain why using `mergeMap` for a search typeahead would be wrong (race conditions, out-of-order results)

Red flag: Uses `mergeMap` everywhere. Does not know that `switchMap` cancels. Cannot name a use case for `exhaustMap`.

Gate 3 | Q22: Scenario — Memory leaks with RxJS in Angular

Question: A developer notices that navigating back and forth between two pages causes memory usage to grow continuously. They are using several Observables in the components. What is happening and how do you prevent it?

Expected Answer:

The most likely cause is **subscriptions that are never unsubscribed** when the component is destroyed. Each navigation creates a new component instance with new subscriptions, but the old subscriptions keep running.

Prevention strategies (from most to least recommended):

```
// Strategy 1: async pipe (best — Angular manages the subscription lifecycle)
@Component({
  template: `
    <div *ngIf="user$ | async as user">
      {{ user.name }}
    </div>
  `,
})
export class ProfileComponent {
  user$ = this.userService.getCurrentUser();
}

// Strategy 2: takeUntilDestroyed (Angular 16+ — cleanest for imperative code)
@Component({ /* ... */ })
export class ProfileComponent {
  private destroyRef = inject(DestroyRef);

  ngOnInit() {
    this.dataService.getData().pipe(
      takeUntilDestroyed(this.destroyRef),
    ).subscribe(data => this.processData(data));
  }
}

// Strategy 3: takeUntil with destroy Subject (pre-Angular 16)
@Component({ /* ... */ })
export class ProfileComponent implements OnDestroy {
  private destroy$ = new Subject<void>();

  ngOnInit() {
    this.dataService.getData().pipe(
      takeUntil(this.destroy$),
    ).subscribe(data => this.processData(data));
  }
```

```
  ngOnDestroy() {
    this.destroy$.next();
    this.destroy$.complete();
  }
}

// Strategy 4: Explicit unsubscribe (most manual, error-prone)
@Component({ /* ... */ })
export class ProfileComponent implements OnDestroy {
  private subscriptions: Subscription[] = [];

  ngOnInit() {
    this.subscriptions.push(
      this.dataService.getData().subscribe(data => this.processData(data)),
    );
  }

  ngOnDestroy() {
    this.subscriptions.forEach(sub => sub.unsubscribe());
  }
}
```

Notable exceptions that do NOT need manual cleanup:

- `HttpClient` observables (they complete after the response)
- `ActivatedRoute.params` / `.data` (Angular manages these)
- Finite observables that complete on their own (e.g., `of()`, `from()`, `timer()` without repeat)

What to listen for:

- Immediately identifies missing unsubscribe as the cause
- Recommends `async` pipe as the primary solution
- Knows about `takeUntilDestroyed` (Angular 16+)
- Knows which observables auto-complete and do not need cleanup

Red flag: Only knows the manual `Subscription` approach. Does not mention `async` pipe. Cannot explain why `HttpClient` observables do not cause leaks.

Gate 4 | Q23: Design a reactive state management solution without NgRx

Question: Your team wants to manage complex UI state (filters, pagination, sorting, selected items) for a data grid without adding NgRx. Design a service-based solution using RxJS that is testable, supports undo, and handles concurrent updates safely.

Expected Answer:

```
// Generic state management service
interface GridState<T> {
  items: T[];
  filters: Record<string, any>;
```

```
  sort: { column: string; direction: 'asc' | 'desc' } | null;
  pagination: { page: number; pageSize: number; total: number };
  selectedIds: Set<string>;
  loading: boolean;
  error: string | null;
}

@Injectable()
export class GridStateService<T extends { id: string }> {
  private stateSubject: BehaviorSubject<GridState<T>>;
  private history: GridState<T>[] = [];
  private maxHistory = 20;

  // Public read-only observable
  readonly state$: Observable<GridState<T>>;

  // Derived selectors (memoized via shareReplay)
  readonly items$ = this.state$.pipe(map(s => s.items), distinctUntilChanged());
  readonly loading$ = this.state$.pipe(map(s => s.loading),
distinctUntilChanged());
  readonly pagination$ = this.state$.pipe(map(s => s.pagination),
distinctUntilChanged());
  readonly selectedItems$ = this.state$.pipe(
    map(s => s.items.filter(item => s.selectedIds.has(item.id))),
    distinctUntilChanged(),
  );

  constructor() {
    const initialState: GridState<T> = {
      items: [],
      filters: {},
      sort: null,
      pagination: { page: 1, pageSize: 25, total: 0 },
      selectedIds: new Set(),
      loading: false,
      error: null,
    };
    this.stateSubject = new BehaviorSubject(initialState);
    this.state$ = this.stateSubject.asObservable().pipe(shareReplay(1));
  }

  // State update with immutability and history tracking
  private updateState(updater: (current: GridState<T>) => Partial<GridState<T>>):
void {
    const current = this.stateSubject.getValue();
    this.history.push(structuredClone(current));
    if (this.history.length > this.maxHistory) {
      this.history.shift();
    }
    const partial = updater(current);
    this.stateSubject.next({ ...current, ...partial });
  }

  // Public actions
```

```
  setFilters(filters: Record<string, any>): void {
    this.updateState(() => ({
      filters,
      pagination: { ...this.stateSubject.getValue().pagination, page: 1 },
    }));
  }

  setSort(column: string, direction: 'asc' | 'desc'): void {
    this.updateState(() => ({ sort: { column, direction } }));
  }

  setPage(page: number): void {
    this.updateState(current => ({
      pagination: { ...current.pagination, page },
    }));
  }

  toggleSelection(id: string): void {
    this.updateState(current => {
      const selectedIds = new Set(current.selectedIds);
      if (selectedIds.has(id)) {
        selectedIds.delete(id);
      } else {
        selectedIds.add(id);
      }
      return { selectedIds };
    });
  }

  // Undo support
  undo(): boolean {
    const previous = this.history.pop();
    if (previous) {
      this.stateSubject.next(previous);
      return true;
    }
    return false;
  }

  // Connect to data source — handles concurrent updates via switchMap
  loadData(dataFetcher: (state: GridState<T>) => Observable<{ items: T[]; total:
number }>): Observable<void> {
    return this.state$.pipe(
      // Only react to changes that affect the query (not selection, not loading)
      map(s => ({ filters: s.filters, sort: s.sort, pagination: s.pagination })),
      distinctUntilChanged((a, b) => JSON.stringify(a) === JSON.stringify(b)),
      switchMap(query => {
        this.stateSubject.next({ ...this.stateSubject.getValue(), loading: true,
error: null });
        return dataFetcher(this.stateSubject.getValue()).pipe(
          catchError(err => {
            this.stateSubject.next({
              ...this.stateSubject.getValue(),
              loading: false,
```

```
                error: err.message,
              });
              return EMPTY;
            }),
          );
        }),
        tap(result => {
          this.updateState(current => ({
            items: result.items,
            loading: false,
            pagination: { ...current.pagination, total: result.total },
          }));
        }),
        map(() => void 0),
      );
    }
  }
```

Design decisions:

- **BehaviorSubject** provides synchronous access to current state and emits the latest value to new subscribers.
- `distinctUntilChanged` on selectors prevents unnecessary re-renders.
- `switchMap` in `loadData` cancels in-flight requests when filters/sort/page changes.
- **Immutable updates** (`{ ...current, ...partial }`) make change detection predictable with `OnPush`.
- **History array** enables undo without a full event-sourcing system.
- **Testable**: inject the service, call actions, assert on emitted state.

What to listen for:

- Uses `BehaviorSubject` (not `Subject` or `ReplaySubject`) for state — knows why
- Implements derived selectors with `distinctUntilChanged` for performance
- Uses `switchMap` for data fetching to cancel stale requests
- Considers immutability for Angular change detection
- Addresses the undo requirement with a bounded history
- Discusses when this approach breaks down and NgRx becomes warranted (many feature stores, complex side effects, time-travel debugging needs)

Red flag: Uses mutable state updates. Does not consider cancellation of in-flight requests. Undo implementation does not bound the history size.

C5. Forms

Gate 2 | Q24: Template-driven vs Reactive Forms — when to use which?

Question: Compare template-driven and reactive forms. Build a reactive form with a custom async validator.

Expected Answer:

Aspect	Template-driven	Reactive
Setup	`FormsModule`, `ngModel`	`ReactiveFormsModule`, `FormGroup`
Logic location	Template	Component class
Testability	Hard (requires DOM)	Easy (pure class testing)
Dynamic forms	Difficult	Straightforward
Validation	Directive-based	Function-based
Best for	Simple forms (login, search)	Complex forms (multi-step, dynamic fields)

```
// Reactive form with custom async validator
@Component({
  selector: 'app-registration',
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]="form" (ngSubmit)="onSubmit()">
      <input formControlName="username" />
      <div *ngIf="form.get('username')?.errors?.['usernameTaken']">
        Username is already taken
      </div>
      <div *ngIf="form.get('username')?.pending">
        Checking availability...
      </div>

      <input formControlName="email" type="email" />
      <input formControlName="password" type="password" />
      <input formControlName="confirmPassword" type="password" />

      <div *ngIf="form.errors?.['passwordMismatch']">
        Passwords do not match
      </div>

      <button [disabled]="form.invalid || form.pending">Register</button>
    </form>
  `,
})
export class RegistrationComponent implements OnInit {
  private fb = inject(FormBuilder);
  private userService = inject(UserService);

  form!: FormGroup;

  ngOnInit() {
    this.form = this.fb.group({
      username: ['', [Validators.required, Validators.minLength(3)],
                     [this.usernameAvailableValidator()]],
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required, Validators.minLength(8)]],
      confirmPassword: ['', [Validators.required]],
```

```
    }, {
      validators: [this.passwordMatchValidator],
    });
  }

  // Custom async validator — checks username availability
  private usernameAvailableValidator(): AsyncValidatorFn {
    return (control: AbstractControl): Observable<ValidationErrors | null> => {
      if (!control.value) return of(null);

      return this.userService.checkUsername(control.value).pipe(
        debounceTime(300),
        map(isTaken => isTaken ? { usernameTaken: true } : null),
        catchError(() => of(null)),
      );
    };
  }

  // Custom cross-field validator
  private passwordMatchValidator(group: AbstractControl): ValidationErrors | null
{
    const password = group.get('password')?.value;
    const confirm = group.get('confirmPassword')?.value;
    return password === confirm ? null : { passwordMismatch: true };
  }

  onSubmit() {
    if (this.form.valid) {
      console.log(this.form.value);
    }
  }
}
```

What to listen for:

- Knows when to use each type (not "always reactive")
- Can build a cross-field validator (password match)
- Implements async validator with debounce and error handling
- Uses `form.pending` to show loading state during async validation

Red flag: Cannot explain the difference. Does not know how to create custom validators. Puts validation logic in the template with template-driven forms and calls it "reactive."

C6. Change Detection

Gate 3 | Q25: Scenario — Performance debugging with change detection

Question: An Angular app becomes sluggish as the data grows. The profiler shows that change detection is running thousands of times per second. The developer is using Default change detection everywhere. Walk through your optimization strategy.

Expected Answer:

Step 1: Understand the problem. Default change detection checks every component in the tree on every browser event (click, keydown, timer, HTTP response). With many components and complex templates, this is expensive.

Step 2: Apply OnPush change detection.

```
@Component({
  selector: 'app-data-row',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <div>{{ item.name }} - {{ item.value | currency }}</div>
  `,
})
export class DataRowComponent {
  @Input() item!: DataItem;
}
```

With `OnPush`, Angular only checks the component when:

1. An `@Input()` reference changes (not deep mutations)
2. An event originates from this component or its children
3. An Observable bound via `async` pipe emits
4. `ChangeDetectorRef.markForCheck()` is called explicitly

Step 3: Ensure immutable data patterns. OnPush compares references, not values:

```
// BAD — mutates the array, OnPush won't detect it
this.items.push(newItem);

// GOOD — new array reference
this.items = [...this.items, newItem];
```

Step 4: Use `trackBy` with `*ngFor`:

```
// template
<div *ngFor="let item of items; trackBy: trackByFn">
  <app-data-row [item]="item"></app-data-row>
</div>

// component
trackByFn(index: number, item: DataItem): string {
  return item.id;
}
```

Without `trackBy`, Angular destroys and recreates all DOM elements when the array reference changes.

Step 5: Angular Signals (Angular 16+):

```
@Component({
  selector: 'app-data-grid',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <div>Total: {{ total() }}</div>
    <div *ngFor="let item of items(); trackBy: trackByFn">
      <app-data-row [item]="item"></app-data-row>
    </div>
  `,
})
export class DataGridComponent {
  items = signal<DataItem[]>([]);
  total = computed(() => this.items().reduce((sum, item) => sum + item.value, 0));

  addItem(item: DataItem) {
    this.items.update(current => [...current, item]);
  }
}
```

Signals provide fine-grained reactivity — Angular knows exactly which parts of the template to update.

Step 6: Additional optimizations:

- Virtual scrolling (`@angular/cdk/scrolling`) for large lists
- Detach change detection entirely for components that update on a timer (`ChangeDetectorRef.detach()`)
- Move heavy computation into `computed()` (signals) or memoized selectors (pure pipes)

What to listen for:

- Starts with `OnPush` as the primary lever
- Understands the 4 triggers for OnPush change detection
- Mentions immutable data patterns as a prerequisite for OnPush
- Knows `trackBy` and can explain why it matters
- Mentions Signals as the modern evolution
- Discusses virtual scrolling for render-bound performance issues

Red flag: Suggests `ChangeDetectorRef.detectChanges()` everywhere (manual override, defeats the purpose). Does not know what triggers OnPush updates. Unaware of Signals.

C7. Lifecycle Hooks

Gate 1 | Q26: List the Angular lifecycle hooks in order and explain when to use the 3 most important ones

Question: What are the Angular component lifecycle hooks, and which ones do you use most often?

Expected Answer:

Lifecycle hooks in order:

1. `constructor` — dependency injection (not technically a lifecycle hook)
2. `ngOnChanges` — called when any `@Input()` property changes (before `ngOnInit` on first run)
3. `ngOnInit` — called once after the first `ngOnChanges`
4. `ngDoCheck` — called on every change detection cycle
5. `ngAfterContentInit` — after content projection (`<ng-content>`) is initialized
6. `ngAfterContentChecked` — after every check of projected content
7. `ngAfterViewInit` — after the component's view (and child views) are initialized
8. `ngAfterViewChecked` — after every check of the component's view
9. `ngOnDestroy` — just before the component is destroyed

The 3 most important:

```
export class ExampleComponent implements OnInit, OnChanges, OnDestroy {
  @Input() userId!: number;
  private destroy$ = new Subject<void>();

  // ngOnInit — one-time initialization logic
  // Use for: fetching initial data, setting up subscriptions
  // Why not constructor? Constructor is for DI only. ngOnInit runs after inputs
are set.
  ngOnInit(): void {
    this.loadUserData(this.userId);
    this.notificationService.messages$.pipe(
      takeUntil(this.destroy$),
    ).subscribe(msg => this.handleMessage(msg));
  }

  // ngOnChanges — react to input changes
  // Use for: re-fetching data when a parent passes a new value
  ngOnChanges(changes: SimpleChanges): void {
    if (changes['userId'] && !changes['userId'].firstChange) {
      this.loadUserData(changes['userId'].currentValue);
    }
  }

  // ngOnDestroy — cleanup
  // Use for: unsubscribing, clearing timers, disconnecting WebSockets
  ngOnDestroy(): void {
    this.destroy$.next();
    this.destroy$.complete();
  }
}
```

What to listen for:

- Knows the order (at least the major hooks)

- Explains why `ngOnInit` over `constructor` for initialization (inputs are not set in constructor)
- Uses `ngOnDestroy` for cleanup
- Knows that `ngOnChanges` runs before `ngOnInit` on first render

C8. HTTP Client and Interceptors

Gate 2 | Q27: Implement an HTTP interceptor for auth tokens and error handling

Question: Show how to implement an interceptor that attaches auth tokens and handles 401 errors with token refresh.

Expected Answer:

```
// Angular 15+ functional interceptor (preferred)
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  // Skip token for public endpoints
  const publicUrls = ['/api/auth/login', '/api/auth/refresh'];
  if (publicUrls.some(url => req.url.includes(url))) {
    return next(req);
  }

  const token = authService.getAccessToken();

  // Clone request with auth header
  const authReq = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  return next(authReq).pipe(
    catchError((error: HttpErrorResponse) => {
      if (error.status === 401) {
        return authService.refreshToken().pipe(
          switchMap(newToken => {
            const retryReq = req.clone({
              setHeaders: { Authorization: `Bearer ${newToken}` },
            });
            return next(retryReq);
          }),
          catchError(refreshError => {
            authService.logout();
            router.navigate(['/login']);
            return throwError(() => refreshError);
          }),
        );
      }
      return throwError(() => error);
    }),
  );
```

```
};

// Registration (in app.config.ts or providers)
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(withInterceptors([authInterceptor, loggingInterceptor])),
  ],
};
```

Key design decisions:

- Skip auth header for public endpoints (login, refresh)
- On 401, attempt token refresh before giving up
- If refresh fails, logout and redirect to login
- Use `switchMap` so only one refresh attempt runs at a time
- Order matters: interceptors run in the order they are listed

A production implementation should also handle the **concurrent refresh** problem (multiple 401s triggering multiple refresh calls). This is typically solved with a shared `BehaviorSubject` or `shareReplay` on the refresh observable.

What to listen for:

- Uses functional interceptor style (Angular 15+)
- Handles token refresh on 401
- Mentions the concurrent refresh problem
- Skips auth for public routes
- Registers via `provideHttpClient(withInterceptors(...))`

Red flag: Uses class-based interceptor without knowing the functional alternative. Does not handle token refresh. Does not consider concurrent 401s.

Gate 3 | Q28: Scenario — API error handling strategy

Question: Design an application-wide error handling strategy for HTTP calls in Angular. Consider: user-facing errors, logging, retry logic, and offline scenarios.

Expected Answer:

Layer the strategy across interceptors, services, and components:

```
// Layer 1: Retry interceptor (handles transient failures)
export const retryInterceptor: HttpInterceptorFn = (req, next) => {
  // Only retry idempotent methods
  const retryable = ['GET', 'HEAD', 'OPTIONS', 'PUT', 'DELETE'];
  if (!retryable.includes(req.method)) {
    return next(req);
  }
```

```
  return next(req).pipe(
    retry({
      count: 2,
      delay: (error, retryCount) => {
        if (error.status === 0 || error.status >= 500) {
          // Exponential backoff: 1s, 2s
          return timer(retryCount * 1000);
        }
        // Don't retry client errors (4xx)
        return throwError(() => error);
      },
    }),
  );
};

// Layer 2: Error normalization interceptor
export const errorNormalizationInterceptor: HttpInterceptorFn = (req, next) => {
  const errorHandler = inject(ErrorHandlerService);

  return next(req).pipe(
    catchError((error: HttpErrorResponse) => {
      const normalized: AppError = {
        status: error.status,
        message: getHumanReadableMessage(error),
        technical: error.message,
        url: error.url ?? req.url,
        timestamp: new Date(),
      };

      // Log all errors
      errorHandler.logError(normalized);

      return throwError(() => normalized);
    }),
  );
};

function getHumanReadableMessage(error: HttpErrorResponse): string {
  if (error.status === 0) return 'Unable to connect. Please check your internet
connection.';
  if (error.status === 403) return 'You do not have permission to perform this
action.';
  if (error.status === 404) return 'The requested resource was not found.';
  if (error.status === 422) return error.error?.message ?? 'The submitted data is
invalid.';
  if (error.status >= 500) return 'A server error occurred. Please try again
later.';
  return 'An unexpected error occurred.';
}

// Layer 3: Service-level error handling
@Injectable({ providedIn: 'root' })
export class UserService {
  private http = inject(HttpClient);
```

```
  getUser(id: number): Observable<User> {
    return this.http.get<User>(`/api/users/${id}`).pipe(
      // Service can add domain-specific handling
      catchError(error => {
        if (error.status === 404) {
          return of(null as unknown as User); // User not found is expected
        }
        return throwError(() => error); // Re-throw everything else
      }),
    );
  }
}

// Layer 4: Component-level — user-facing presentation
@Component({ /* ... */ })
export class UserProfileComponent {
  error = signal<string | null>(null);

  loadUser(id: number) {
    this.userService.getUser(id).subscribe({
      next: user => this.user.set(user),
      error: err => this.error.set(err.message),
    });
  }
}

// Layer 5: Global error handler for unhandled errors
@Injectable()
export class GlobalErrorHandler implements ErrorHandler {
  private errorService = inject(ErrorHandlerService);

  handleError(error: unknown): void {
    this.errorService.logError({
      message: error instanceof Error ? error.message : 'Unknown error',
      technical: error instanceof Error ? error.stack ?? '' : String(error),
      timestamp: new Date(),
    });
    // Show toast/snackbar for unhandled errors
  }
}
```

Interceptor ordering: `retryInterceptor` -> `authInterceptor` -> `errorNormalizationInterceptor`

Retry runs first so retries happen before error normalization. Auth interceptor refreshes tokens before errors are normalized. Error normalization is last so it processes the final error state.

What to listen for:

- Designs a layered approach (interceptor, service, component, global handler)
- Differentiates between retryable and non-retryable requests
- Provides human-readable error messages (not raw HTTP errors to users)

- Considers the interceptor ordering carefully
- Mentions offline detection or `status === 0`

Red flag: Puts all error handling in one interceptor. Shows error.message from the server directly to the user. Does not consider retry logic or offline scenarios. Retries POST requests blindly.

C9. State Management

Gate 3 | Q29: Scenario — Choosing a state management approach

Question: Your team is starting a new Angular project. Some developers want NgRx, others want a simple service-based approach. The app has: a product catalog, a shopping cart, user authentication, and an admin dashboard. Advise the team.

Expected Answer:

This is a "right tool for the right job" situation. Not all state is equal:

State type	Scope	Recommended approach
User auth (token, role)	Global, rarely changes	Simple service with `BehaviorSubject`
Shopping cart	Global, moderate updates	NgRx ComponentStore or signal-based service
Product catalog	Feature-scoped, server-driven	HTTP cache + service (or Angular query/TanStack)
Admin dashboard filters	Local, component-scoped	Reactive forms or component signals

```
// Auth — simple service (overkill for NgRx)
@Injectable({ providedIn: 'root' })
export class AuthStore {
  private userSubject = new BehaviorSubject<User | null>(null);
  user$ = this.userSubject.asObservable();
  isAuthenticated$ = this.user$.pipe(map(u => u !== null));

  login(credentials: Credentials): Observable<void> {
    return this.authApi.login(credentials).pipe(
      tap(response => this.userSubject.next(response.user)),
      map(() => void 0),
    );
  }

  logout(): void {
    this.userSubject.next(null);
    this.tokenService.clearTokens();
  }
}

// Shopping Cart — ComponentStore (medium complexity, isolated state)
```

```
@Injectable()
export class CartStore extends ComponentStore<CartState> {
  constructor() {
    super({ items: [], coupon: null });
  }

  // Selectors
  readonly items$ = this.select(state => state.items);
  readonly total$ = this.select(state =>
    state.items.reduce((sum, item) => sum + item.price * item.quantity, 0),
  );
  readonly itemCount$ = this.select(state =>
    state.items.reduce((sum, item) => sum + item.quantity, 0),
  );

  // Updaters
  readonly addItem = this.updater((state, item: CartItem) => ({
    ...state,
    items: [...state.items, item],
  }));

  readonly removeItem = this.updater((state, productId: string) => ({
    ...state,
    items: state.items.filter(i => i.productId !== productId),
  }));

  // Effects (async operations)
  readonly checkout = this.effect((trigger$: Observable<void>) =>
    trigger$.pipe(
      withLatestFrom(this.state$),
      switchMap(([, state]) => this.cartApi.submitOrder(state.items)),
    ),
  );
}
```

Decision framework:

- **Simple service** when: 1-2 consumers, straightforward CRUD, no complex derived state
- **ComponentStore** when: feature-scoped state, moderate complexity, team does not want full NgRx
- **NgRx Store** when: many features share state, need devtools/time-travel, complex side effects across features, large team needing strict patterns
- **Signals** (Angular 16+) when: replacing BehaviorSubject in simple-to-medium cases, want framework-native reactivity

What to listen for:

- Does not give a one-size-fits-all answer
- Categorizes state by scope and complexity
- Knows NgRx ComponentStore as a middle ground
- Mentions Signals as the emerging alternative
- Considers team familiarity and project lifespan

Red flag: Insists on NgRx for everything (or refuses NgRx entirely). Does not know that ComponentStore exists. Cannot explain what problem NgRx solves that services do not.

C10. Performance

Gate 3 | Q30: Scenario — Bundle size and load time optimization

Question: Your Angular app takes 8 seconds to load on a 3G connection. The main bundle is 1.2MB. Walk through your optimization approach.

Expected Answer:

Step 1: Analyze the bundle.

```
# Generate a bundle analysis report
ng build --stats-json
npx webpack-bundle-analyzer dist/my-app/stats.json
```

Look for: large third-party libraries, duplicate imports, unused code.

Step 2: Implement lazy loading.

```
// Before: eagerly loaded in AppModule
import { AdminModule } from './admin/admin.module';

// After: lazy loaded
const routes: Routes = [
  {
    path: 'admin',
    loadChildren: () => import('./admin/admin.routes').then(m => m.ADMIN_ROUTES),
  },
  {
    path: 'reports',
    loadComponent: () => import('./reports/reports.component').then(m =>
m.ReportsComponent),
  },
];
```

Step 3: Tree-shake third-party imports.

```
// BAD — imports entire lodash (600KB+)
import _ from 'lodash';
_.debounce(fn, 300);

// GOOD — imports only what's needed (~4KB)
import debounce from 'lodash-es/debounce';
debounce(fn, 300);
```

```
// Even better — use native or lightweight alternatives
// RxJS debounceTime, native Array methods, etc.
```

Step 4: Optimize images and assets. Use `loading="lazy"` on images, modern formats (WebP, AVIF), CDN.

Step 5: Enable production optimizations.

```
// angular.json (usually enabled by default in production)
{
  "configurations": {
    "production": {
      "optimization": true,
      "aot": true,
      "buildOptimizer": true,
      "sourceMap": false,
      "extractCss": true
    }
  }
}
```

Step 6: Runtime performance.

- Preload lazy modules with `PreloadAllModules` or a custom strategy
- Use service worker for caching (`@angular/pwa`)
- Server-Side Rendering (SSR) or Static Site Generation (SSG) for initial load

Measurement:

- Lighthouse audit (Core Web Vitals: LCP, FID, CLS)
- `ng build` output size comparison before/after each optimization
- Real User Monitoring (RUM) to validate improvement in production

What to listen for:

- Starts with measurement/analysis, not guessing
- Lazy loading is the first concrete action
- Knows about tree-shaking lodash and similar libraries
- Mentions bundle analyzer as a diagnostic tool
- Considers both transfer size (gzip) and parse time
- Has a measurement strategy (Lighthouse, RUM)

Red flag: Suggests "upgrade the server" for a client-side rendering issue. Does not know about lazy loading. Cannot explain tree-shaking.

C11. Angular CLI and Build Configuration

Gate 1 | Q31: What are the most important Angular CLI commands you use daily?

Question: Walk through your typical Angular CLI usage.

Expected Answer:

```
# Project creation
ng new my-app --standalone --routing --style=scss

# Generate components, services, etc.
ng generate component features/dashboard --standalone
ng generate service core/services/auth
ng generate guard core/guards/auth --functional
ng generate pipe shared/pipes/currency
ng generate interceptor core/interceptors/auth --functional

# Development
ng serve                      # dev server on localhost:4200
ng serve --port 4300          # custom port
ng serve --configuration=staging  # use staging environment

# Build
ng build                      # development build
ng build --configuration=production  # production build

# Testing
ng test                       # run unit tests (Karma)
ng test --code-coverage       # with coverage report
ng test --watch=false         # single run (CI)
ng e2e                        # end-to-end tests

# Linting
ng lint                       # ESLint

# Updates
ng update @angular/core @angular/cli  # update Angular
```

What to listen for:

- Uses `ng generate` rather than creating files manually
- Knows the `--standalone` flag for new components
- Understands build configurations (production vs staging)
- Mentions `ng update` for Angular version upgrades

C12. Signals (Angular 16+)

Gate 2 | Q32: What are Angular Signals and how do they compare to RxJS Observables?

Question: Explain Angular Signals introduced in Angular 16. When would you use Signals vs RxJS?

Expected Answer:

Signals are a **synchronous, fine-grained reactivity primitive** built into Angular. They track dependencies automatically and notify consumers when their value changes.

```
import { signal, computed, effect } from '@angular/core';

// Writable signal
const count = signal(0);

// Read the value
console.log(count()); // 0

// Update the value
count.set(5);
count.update(current => current + 1); // 6

// Computed signal (derived, read-only, automatically tracks dependencies)
const doubled = computed(() => count() * 2); // 12

// Effect (side effect that runs when dependencies change)
effect(() => {
  console.log(`Count changed to: ${count()}`);
});

// In a component
@Component({
  selector: 'app-counter',
  template: `
    <p>Count: {{ count() }}</p>
    <p>Doubled: {{ doubled() }}</p>
    <button (click)="increment()">+1</button>
  `,
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class CounterComponent {
  count = signal(0);
  doubled = computed(() => this.count() * 2);

  increment() {
    this.count.update(c => c + 1);
  }
}
```

Signals vs RxJS:

Aspect	Signals	RxJS Observables
Synchronous/Async	Synchronous	Async (push-based)
Current value	Always available (`signal()`)	Not always (`BehaviorSubject` has it)
Cleanup	Automatic	Manual (unsubscribe)

Aspect	Signals	RxJS Observables
Operators	Limited (computed)	Rich operator library
Best for	UI state, derived values	Events, HTTP, WebSockets, complex async flows
Change detection	Fine-grained (only re-render affected parts)	Zone-based or `async` pipe

When to use each:

- **Signals**: component state, form values, UI toggles, computed values, anything synchronous
- **RxJS**: HTTP calls, event streams, WebSocket messages, complex async composition (debounce, switchMap, merge)
- **Together**: use `toSignal()` and `toObservable()` to bridge between them

```
// Convert Observable to Signal
const user = toSignal(this.userService.currentUser$, { initialValue: null });

// Convert Signal to Observable
const count$ = toObservable(this.count);
```

What to listen for:

- Understands that Signals are synchronous and always have a current value
- Knows the `toSignal` / `toObservable` interop functions
- Does not say "Signals replace RxJS" — they complement each other
- Mentions fine-grained change detection as the main benefit

Red flag: Thinks Signals completely replace RxJS. Does not know about `computed`. Cannot explain when Signals are better than BehaviorSubject.

C13. Component Communication Patterns

Gate 3 | Q33: Scenario — Deep component tree communication

Question: You have a deeply nested component tree (5+ levels). A child component at the bottom needs data from the top-level component and also needs to send events back up. An intermediate developer is passing `@Input`/`@Output` through every level (prop drilling). What alternatives would you suggest?

Expected Answer:

Prop drilling creates maintenance burden and couples intermediate components to data they do not use. Alternatives:

```
// Option 1: Shared service with Observable/Signal (most common in Angular)
@Injectable({ providedIn: 'root' }) // or provided at a shared ancestor
```

```
export class SelectionService {
  private selectedItem = signal<Item | null>(null);

  readonly selectedItem$ = this.selectedItem.asReadonly();

  select(item: Item): void {
    this.selectedItem.set(item);
  }

  clear(): void {
    this.selectedItem.set(null);
  }
}

// Top-level component
@Component({
  template: `<app-item-list></app-item-list>
             <app-detail-panel [item]="selectionService.selectedItem$()" />`,
})
export class PageComponent {
  selectionService = inject(SelectionService);
}

// Deep child component (no @Input chain needed)
@Component({
  template: `<button (click)="onSelect()">Select</button>`,
})
export class DeepChildComponent {
  private selectionService = inject(SelectionService);
  @Input() item!: Item;

  onSelect() {
    this.selectionService.select(this.item);
  }
}

// Option 2: Content projection for layout concerns
@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      <div class="card-header"><ng-content select="[card-header]"></ng-content>
</div>
      <div class="card-body"><ng-content></ng-content></div>
    </div>
  `,
})
export class CardComponent { }

// Usage — parent passes content directly, no intermediate data passing
<app-card>
  <h2 card-header>{{ title }}</h2>
  <app-deep-child [data]="data" (action)="onAction($event)"></app-deep-child>
</app-card>
```

```
// Option 3: Provide at ancestor level (hierarchical DI)
// Provide the service at the feature component level so it's scoped to that
subtree
@Component({
  providers: [SelectionService], // new instance for this subtree only
  template: `<app-nested-tree></app-nested-tree>`,
})
export class FeatureComponent { }
```

When to use each:

- **Shared service**: most common, works for any depth, supports both data and events
- **Content projection**: when the intermediate components are just layout wrappers
- **Hierarchical DI**: when you need a scoped instance per feature area (e.g., each tab has its own state)
- **NgRx/Signals store**: when many unrelated components need the same state

What to listen for:

- Identifies prop drilling as the problem, not a solution
- Proposes multiple alternatives with tradeoffs
- Mentions hierarchical DI scoping (provide at ancestor level)
- Considers content projection for layout cases
- Does not jump straight to NgRx for a simple communication problem

Red flag: Only knows `@Input`/`@Output`. Suggests event emitters that bubble through every intermediate component. Proposes NgRx for a 2-component communication problem.

C14. Testing Angular Components and Services

Gate 2 | Q34: Write a unit test for an Angular service that calls an API

Question: Write tests for this service method:

```
@Injectable({ providedIn: 'root' })
export class UserService {
  private http = inject(HttpClient);

  getUser(id: number): Observable<User> {
    return this.http.get<User>(`/api/users/${id}`);
  }
}
```

Expected Answer:

```
describe('UserService', () => {
  let service: UserService;
```

```
  let httpMock: HttpTestingController;

  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [HttpClientTestingModule],
    });
    service = TestBed.inject(UserService);
    httpMock = TestBed.inject(HttpTestingController);
  });

  afterEach(() => {
    httpMock.verify(); // Ensure no outstanding requests
  });

  it('should fetch a user by ID', () => {
    const mockUser: User = { id: 1, name: 'Alice', email: 'alice@example.com' };

    service.getUser(1).subscribe(user => {
      expect(user).toEqual(mockUser);
    });

    const req = httpMock.expectOne('/api/users/1');
    expect(req.request.method).toBe('GET');
    req.flush(mockUser);
  });

  it('should handle 404 error', () => {
    service.getUser(999).subscribe({
      next: () => fail('Expected an error'),
      error: (error: HttpErrorResponse) => {
        expect(error.status).toBe(404);
      },
    });

    const req = httpMock.expectOne('/api/users/999');
    req.flush('Not Found', { status: 404, statusText: 'Not Found' });
  });

  it('should handle network error', () => {
    service.getUser(1).subscribe({
      next: () => fail('Expected an error'),
      error: (error: HttpErrorResponse) => {
        expect(error.status).toBe(0);
      },
    });

    const req = httpMock.expectOne('/api/users/1');
    req.error(new ProgressEvent('Network error'));
  });
});
```

Key testing patterns:

- `HttpClientTestingModule` provides `HttpTestingController` for mocking HTTP requests
- `httpMock.expectOne()` asserts that exactly one request was made to the URL
- `req.flush()` provides the mock response
- `httpMock.verify()` in `afterEach` ensures no unexpected requests
- Test both success and error paths

What to listen for:

- Uses `HttpTestingController` (not manually mocking HttpClient)
- Includes `httpMock.verify()` in `afterEach`
- Tests error scenarios (404, network error)
- Uses `fail()` in the success callback of error tests to catch false positives

Red flag: Manually mocks HttpClient instead of using the testing module. Does not test error paths. Forgets `httpMock.verify()`.

Gate 3 | Q35: Scenario — Testing a component with complex dependencies

Question: How would you test a component that depends on `ActivatedRoute`, a service, and uses `Router` for navigation? The component reads a route parameter and loads data.

Expected Answer:

```
@Component({
  selector: 'app-user-profile',
  template: `
    <div *ngIf="user">{{ user.name }}</div>
    <button (click)="goBack()">Back</button>
  `,
})
export class UserProfileComponent implements OnInit {
  user: User | null = null;
  private route = inject(ActivatedRoute);
  private router = inject(Router);
  private userService = inject(UserService);

  ngOnInit() {
    const id = Number(this.route.snapshot.paramMap.get('id'));
    this.userService.getUser(id).subscribe(user => this.user = user);
  }

  goBack() {
    this.router.navigate(['/users']);
  }
}

// Test
describe('UserProfileComponent', () => {
  let component: UserProfileComponent;
  let fixture: ComponentFixture<UserProfileComponent>;
  let userServiceSpy: jasmine.SpyObj<UserService>;
```

```
  let routerSpy: jasmine.SpyObj<Router>;

  const mockUser: User = { id: 42, name: 'Alice', email: 'alice@example.com' };

  beforeEach(async () => {
    userServiceSpy = jasmine.createSpyObj('UserService', ['getUser']);
    routerSpy = jasmine.createSpyObj('Router', ['navigate']);

    await TestBed.configureTestingModule({
      declarations: [UserProfileComponent],
      providers: [
        { provide: UserService, useValue: userServiceSpy },
        { provide: Router, useValue: routerSpy },
        {
          provide: ActivatedRoute,
          useValue: {
            snapshot: {
              paramMap: convertToParamMap({ id: '42' }),
            },
          },
        },
      ],
    }).compileComponents();

    fixture = TestBed.createComponent(UserProfileComponent);
    component = fixture.componentInstance;
  });

  it('should load user data on init', () => {
    userServiceSpy.getUser.and.returnValue(of(mockUser));

    fixture.detectChanges(); // triggers ngOnInit

    expect(userServiceSpy.getUser).toHaveBeenCalledWith(42);
    expect(component.user).toEqual(mockUser);
  });

  it('should display user name in template', () => {
    userServiceSpy.getUser.and.returnValue(of(mockUser));
    fixture.detectChanges();

    const element = fixture.nativeElement.querySelector('div');
    expect(element.textContent).toContain('Alice');
  });

  it('should navigate back to users list', () => {
    userServiceSpy.getUser.and.returnValue(of(mockUser));
    fixture.detectChanges();

    component.goBack();

    expect(routerSpy.navigate).toHaveBeenCalledWith(['/users']);
  });
```

```
  it('should handle user not found', () => {
    userServiceSpy.getUser.and.returnValue(throwError(() => new Error('Not
found')));

    fixture.detectChanges();

    expect(component.user).toBeNull();
  });
});
```

Key patterns:

- Provide mock `ActivatedRoute` with `convertToParamMap` for realistic param access
- Use `jasmine.createSpyObj` for service and router mocks
- Call `fixture.detectChanges()` to trigger lifecycle hooks
- Test both the component logic and the template rendering
- Test navigation calls with `expect(router.navigate).toHaveBeenCalledWith()`

What to listen for:

- Knows how to mock `ActivatedRoute` (this is a common stumbling point)
- Separates component logic tests from template rendering tests
- Tests the error path
- Calls `fixture.detectChanges()` at the right time

Red flag: Does not know how to provide a mock `ActivatedRoute`. Only tests the happy path. Does not call `fixture.detectChanges()`.

C15. Advanced Patterns

Gate 4 | Q36: Design a micro-frontend architecture with Angular

Question: Your organization wants to break a monolithic Angular app into micro-frontends owned by different teams. Design the architecture covering: module federation, shared dependencies, communication between micro-frontends, routing, and deployment.

Expected Answer:

Architecture: Module Federation with Webpack 5 (or Native Federation for esbuild)

```
                    ┌──────────────────┐
                    │   Shell (Host)   │
                    │  - Routing       │
                    │  - Auth          │
                    │  - Shared UI     │
                    └──────────────────┘
                       │    │    │
              ┌────────┘    │    └────────┐
              ▼             ▼             ▼
```

```
    ┌──────────────┐ ┌──────────────┐ ┌──────────────┐
    │  Product MFE  │ │   Cart MFE   │ │  Admin MFE   │
    │  Team Alpha   │ │  Team Beta   │ │  Team Gamma  │
    └──────────────┘ └──────────────┘ └──────────────┘
```

Shell (Host) Configuration:

```
// webpack.config.js (shell)
const ModuleFederationPlugin = require('@angular-architects/module-
federation/webpack');

module.exports = {
  plugins: [
    new ModuleFederationPlugin({
      remotes: {
        productMfe: 'productMfe@http://cdn.example.com/product/remoteEntry.js',
        cartMfe: 'cartMfe@http://cdn.example.com/cart/remoteEntry.js',
      },
      shared: {
        '@angular/core': { singleton: true, strictVersion: true },
        '@angular/common': { singleton: true, strictVersion: true },
        '@angular/router': { singleton: true, strictVersion: true },
        rxjs: { singleton: true, strictVersion: false },
      },
    }),
  ],
};

// Shell routing
const routes: Routes = [
  {
    path: 'products',
    loadChildren: () => loadRemoteModule({
      type: 'module',
      remoteEntry: 'http://cdn.example.com/product/remoteEntry.js',
      exposedModule: './ProductRoutes',
    }).then(m => m.PRODUCT_ROUTES),
  },
  {
    path: 'cart',
    loadChildren: () => loadRemoteModule({
      type: 'module',
      remoteEntry: 'http://cdn.example.com/cart/remoteEntry.js',
      exposedModule: './CartRoutes',
    }).then(m => m.CART_ROUTES),
  },
];
```

Cross-MFE Communication:

```
// Shared library (@org/mfe-contracts) — published as an npm package
// All MFEs depend on this for type safety

export interface MfeEvent<T = unknown> {
  type: string;
  payload: T;
  source: string;
  timestamp: number;
}

export interface CartUpdatedEvent extends MfeEvent<{ itemCount: number }> {
  type: 'cart:updated';
}

// Event bus service (provided by the shell, injected everywhere)
@Injectable({ providedIn: 'root' })
export class MfeEventBus {
  private events$ = new Subject<MfeEvent>();

  emit(event: MfeEvent): void {
    this.events$.next(event);
  }

  on<T extends MfeEvent>(type: string): Observable<T> {
    return this.events$.pipe(
      filter(e => e.type === type),
    ) as Observable<T>;
  }
}

// Alternative: Custom Events on window (works even without shared DI)
// Useful as a fallback when MFEs use different frameworks
window.dispatchEvent(new CustomEvent('mfe:cart:updated', {
  detail: { itemCount: 5 },
}));
```

Shared Dependencies Strategy:

- Angular core packages: singleton in shell, shared via Module Federation
- Design system / UI library: shared singleton, versioned contract
- Business logic: each MFE owns its own; shared via API contracts, not shared code
- CSS: design tokens (CSS custom properties) shared via CDN; each MFE scopes its own styles

Deployment:

- Each MFE has its own CI/CD pipeline and deploys independently
- Shell loads `remoteEntry.js` at runtime (not build-time)
- Version contract: MFEs can update independently as long as they respect shared dependency version ranges
- Feature flags to control MFE rollout

Tradeoffs:

- **Pro**: team autonomy, independent deployments, technology flexibility
- **Con**: increased complexity, shared dependency version management, harder debugging across boundaries, performance overhead from multiple bundles

What to listen for:

- Proposes Module Federation (not just iframes)
- Addresses shared singleton dependencies (Angular core must be single instance)
- Designs a communication mechanism between MFEs
- Considers independent deployment as a key benefit
- Discusses version compatibility challenges
- Mentions a shared contract library for type safety

Red flag: Proposes iframes as the primary solution. Does not address shared dependencies (loading Angular 3 times). No communication strategy between MFEs. Does not consider deployment independence.

Gate 4 | Q37: Design a real-time collaborative feature using Angular

Question: Design a collaborative document editing feature where multiple users can see each other's changes in real-time. Cover: WebSocket integration, conflict resolution, optimistic updates, and Angular architecture.

Expected Answer:

Architecture Overview:

```
Angular Client A          Server              Angular Client B
     |                      |                       |
     |── edit operation ──>  |                       |
     |   (optimistic UI)    |── broadcast ────────> |
     |                      |   (transformed op)    |── apply to local doc
     |<── ack + transform ──|                       |
     |   (resolve conflicts)|                       |
```

WebSocket Service:

```
@Injectable({ providedIn: 'root' })
export class WebSocketService {
  private socket$ = new ReplaySubject<WebSocket>(1);
  private messagesSubject = new Subject<ServerMessage>();

  readonly messages$ = this.messagesSubject.asObservable();
  readonly connectionState = signal<'connecting' | 'connected' | 'disconnected'>
('disconnected');

  connect(url: string): void {
    this.connectionState.set('connecting');
```

```
    const socket = new WebSocket(url);

    socket.onopen = () => {
      this.connectionState.set('connected');
      this.socket$.next(socket);
    };

    socket.onmessage = (event) => {
      const message = JSON.parse(event.data) as ServerMessage;
      // Run inside NgZone to trigger change detection
      NgZone.assertInAngularZone();
      this.messagesSubject.next(message);
    };

    socket.onclose = () => {
      this.connectionState.set('disconnected');
      // Reconnect with exponential backoff
      this.reconnect(url);
    };
  }

  send(message: ClientMessage): void {
    this.socket$.pipe(take(1)).subscribe(socket => {
      socket.send(JSON.stringify(message));
    });
  }

  private reconnect(url: string, attempt = 1): void {
    const delay = Math.min(1000 * Math.pow(2, attempt), 30000);
    setTimeout(() => this.connect(url), delay);
  }
}
```

Operational Transform (OT) / CRDT for Conflict Resolution:

```
// Simplified OT-based collaborative editing
interface Operation {
  id: string;
  userId: string;
  type: 'insert' | 'delete';
  position: number;
  content?: string; // for insert
  length?: number;  // for delete
  version: number;  // document version this op was based on
}

@Injectable({ providedIn: 'root' })
export class CollaborativeDocService {
  private document = signal<string>('');
  private version = signal<number>(0);
  private pendingOps: Operation[] = []; // sent but not yet acknowledged
  private buffer: Operation[] = [];     // created while pending ops exist
```

```
  readonly document$ = this.document.asReadonly();

  constructor(private ws: WebSocketService) {
    // Listen for remote operations
    this.ws.messages$.pipe(
      filter((msg): msg is RemoteOpMessage => msg.type === 'operation'),
    ).subscribe(msg => this.handleRemoteOperation(msg.operation));

    // Listen for acknowledgements
    this.ws.messages$.pipe(
      filter((msg): msg is AckMessage => msg.type === 'ack'),
    ).subscribe(msg => this.handleAck(msg));
  }

  // Local edit — apply optimistically, then send to server
  applyLocalEdit(op: Omit<Operation, 'id' | 'version' | 'userId'>): void {
    const operation: Operation = {
      ...op,
      id: crypto.randomUUID(),
      userId: this.authService.userId,
      version: this.version(),
    };

    // Apply to local document immediately (optimistic)
    this.applyOperation(operation);

    if (this.pendingOps.length === 0) {
      // No pending ops — send immediately
      this.pendingOps.push(operation);
      this.ws.send({ type: 'operation', operation });
    } else {
      // Buffer until pending op is acknowledged
      this.buffer.push(operation);
    }
  }

  private handleRemoteOperation(remoteOp: Operation): void {
    // Transform remote op against pending and buffered ops
    let transformedOp = remoteOp;
    for (const pending of [...this.pendingOps, ...this.buffer]) {
      transformedOp = this.transform(transformedOp, pending);
    }
    this.applyOperation(transformedOp);
    this.version.update(v => v + 1);
  }

  private handleAck(ack: AckMessage): void {
    this.pendingOps = [];
    this.version.set(ack.version);

    // Send buffered ops
    if (this.buffer.length > 0) {
      const nextOp = this.buffer.shift()!;
```

```
      nextOp.version = this.version();
      this.pendingOps.push(nextOp);
      this.ws.send({ type: 'operation', operation: nextOp });
    }
  }

  private transform(op1: Operation, op2: Operation): Operation {
    // Operational Transform logic
    // If both are inserts and op1's position >= op2's position,
    // shift op1's position by op2's content length
    // (simplified — real OT is more complex)
    if (op1.type === 'insert' && op2.type === 'insert') {
      if (op1.position >= op2.position) {
        return { ...op1, position: op1.position + (op2.content?.length ?? 0) };
      }
    }
    return op1; // Simplified — real implementation handles all combinations
  }

  private applyOperation(op: Operation): void {
    this.document.update(doc => {
      if (op.type === 'insert') {
        return doc.slice(0, op.position) + (op.content ?? '') +
doc.slice(op.position);
      }
      if (op.type === 'delete') {
        return doc.slice(0, op.position) + doc.slice(op.position + (op.length ??
0));
      }
      return doc;
    });
  }
}
```

Presence (cursors and selections):

```
@Injectable({ providedIn: 'root' })
export class PresenceService {
  readonly cursors = signal<Map<string, CursorPosition>>(new Map());

  constructor(private ws: WebSocketService) {
    this.ws.messages$.pipe(
      filter((msg): msg is CursorMessage => msg.type === 'cursor'),
    ).subscribe(msg => {
      this.cursors.update(map => {
        const updated = new Map(map);
        updated.set(msg.userId, msg.position);
        return updated;
      });
    });
  }
```

```
  broadcastCursor(position: CursorPosition): void {
    this.ws.send({ type: 'cursor', position });
  }
}
```

Key design decisions:

- **Optimistic updates**: local edits apply immediately for responsiveness
- **OT vs CRDT**: OT requires a central server for ordering; CRDTs are decentralized but have higher space complexity. For a server-backed Angular app, OT is simpler.
- **Reconnection**: exponential backoff with jitter prevents thundering herd
- **NgZone**: WebSocket messages must run inside Angular's zone for change detection (or use `OnPush` with signals)

What to listen for:

- Mentions OT or CRDT for conflict resolution (not just "last write wins")
- Implements optimistic updates with rollback/transform capability
- Handles reconnection gracefully
- Considers presence (cursor positions) as a separate concern
- Discusses the NgZone implication of WebSocket callbacks

Red flag: Proposes polling instead of WebSockets. Uses "last write wins" without acknowledging data loss. No conflict resolution strategy. Does not consider offline/reconnection scenarios.

C16. Security in Angular

Gate 3 | Q38: Scenario — Securing an Angular application

Question: A security audit found several vulnerabilities in your Angular app: XSS in user-generated content, CSRF exposure, and sensitive data in local storage. How do you address each?

Expected Answer:

XSS Prevention:

```
// Angular auto-sanitizes template bindings — this is SAFE
<div>{{ userContent }}</div> // Automatically HTML-encoded

// DANGEROUS — bypasses sanitization
<div [innerHTML]="userContent"></div> // Angular sanitizes, but be cautious

// If you truly need raw HTML (e.g., from a CMS), sanitize explicitly
@Component({ /* ... */ })
export class ContentComponent {
  private sanitizer = inject(DomSanitizer);

  // Only use bypassSecurityTrust* when you control the source
  getSafeHtml(content: string): SafeHtml {
```

```
    // Prefer using Angular's built-in sanitization
    // Only bypass when content comes from a trusted source (your CMS, not user
input)
    return this.sanitizer.bypassSecurityTrustHtml(content);
  }
}

// Content Security Policy header (server-side)
// Content-Security-Policy: default-src 'self'; script-src 'self'; style-src
'self' 'unsafe-inline'
```

CSRF Prevention:

```
// Angular's HttpClient automatically reads the XSRF-TOKEN cookie
// and sends it as X-XSRF-TOKEN header (for same-origin requests)

// Configure if your backend uses different names:
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withXsrfConfiguration({
        cookieName: 'MY-XSRF-TOKEN',
        headerName: 'X-MY-XSRF-TOKEN',
      }),
    ),
  ],
};

// Important: XSRF protection only works for:
// - Same-origin requests (not cross-origin APIs)
// - Mutating methods (POST, PUT, DELETE — not GET)
// For cross-origin APIs, use auth tokens instead
```

Sensitive Data in Storage:

```
// BAD — tokens in localStorage (accessible via XSS)
localStorage.setItem('auth_token', token);

// BETTER — httpOnly cookies (not accessible via JavaScript)
// Set by the server:
// Set-Cookie: auth_token=xyz; HttpOnly; Secure; SameSite=Strict

// If you must store in the browser (e.g., SPAs with Bearer tokens):
// - Use sessionStorage over localStorage (cleared when tab closes)
// - Never store refresh tokens client-side
// - Implement token rotation
// - Set short expiration times

// For sensitive data that must persist across page loads,
```

```
// consider using the Web Crypto API for encryption:
async function encryptAndStore(key: string, data: string): Promise<void> {
  const encoder = new TextEncoder();
  const cryptoKey = await window.crypto.subtle.generateKey(
    { name: 'AES-GCM', length: 256 },
    true,
    ['encrypt', 'decrypt'],
  );
  const iv = window.crypto.getRandomValues(new Uint8Array(12));
  const encrypted = await window.crypto.subtle.encrypt(
    { name: 'AES-GCM', iv },
    cryptoKey,
    encoder.encode(data),
  );
  sessionStorage.setItem(key, JSON.stringify({
    data: Array.from(new Uint8Array(encrypted)),
    iv: Array.from(iv),
  }));
  // Note: the cryptoKey itself must be stored securely — this is a simplified
example
}
```

Additional security measures:

- Set `Content-Security-Policy` headers
- Use `Strict-Transport-Security` (HSTS) headers
- Validate all input on the server (client-side validation is for UX, not security)
- Implement route guards to prevent unauthorized page access
- Audit third-party dependencies (`npm audit`)

What to listen for:

- Knows Angular's built-in XSS protection (template sanitization)
- Warns against `bypassSecurityTrust*` for user-generated content
- Understands that CSRF tokens work for same-origin requests
- Recommends httpOnly cookies over localStorage for tokens
- Mentions CSP headers as a defense-in-depth measure

Red flag: Uses `bypassSecurityTrustHtml` for user-generated content. Stores tokens in localStorage and says "it's fine." Does not know about CSRF protection built into Angular. Relies solely on client-side validation.

↑ Back to Index

PART D — HTML/CSS Essentials

D1. Flexbox and Grid

Gate 1 | Q39: When do you use Flexbox vs CSS Grid?

Question: Explain the difference between Flexbox and CSS Grid. Give an example of when each is the better choice.

Expected Answer:

Flexbox — one-dimensional layout (row OR column):

```
/* Navigation bar — items in a row */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 1rem;
}

/* Centering content vertically and horizontally */
.hero {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}

/* Card footer pushed to bottom */
.card {
  display: flex;
  flex-direction: column;
  height: 100%;
}
.card-body { flex: 1; }
.card-footer { margin-top: auto; }
```

CSS Grid — two-dimensional layout (rows AND columns):

```
/* Page layout */
.app-layout {
  display: grid;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  grid-template-columns: 250px 1fr;
  grid-template-rows: auto 1fr auto;
  min-height: 100vh;
}
.header  { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main    { grid-area: main; }
.footer  { grid-area: footer; }
```

```
/* Card grid — equal-size items with responsive columns */
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
  gap: 1.5rem;
}
```

When to use which:

Scenario	Best choice
Navbar, toolbar, button group	Flexbox
Centering a single element	Flexbox
Page layout (header, sidebar, content, footer)	Grid
Card grid with equal-size items	Grid
Items of varying size in a row	Flexbox
Complex 2D placement with overlapping	Grid
Aligning items along a single axis	Flexbox

They are not mutually exclusive. Use Grid for the page layout and Flexbox for components within grid cells.

What to listen for:

- Understands 1D vs 2D distinction
- Can give specific use cases for each
- Knows `auto-fill`/`auto-fit` with `minmax` for responsive grids
- Mentions using both together

D2. Responsive Design

Gate 2 | Q40: Implement a responsive layout strategy

Question: How do you approach responsive design? Show a CSS strategy for a layout that works across mobile, tablet, and desktop.

Expected Answer:

Mobile-first approach with CSS custom properties and media queries:

```
/* Design tokens */
:root {
  --spacing-sm: 0.5rem;
  --spacing-md: 1rem;
  --spacing-lg: 2rem;
  --max-width: 1200px;
}
```

```
/* Base styles (mobile-first) */
.container {
  width: 100%;
  max-width: var(--max-width);
  margin: 0 auto;
  padding: 0 var(--spacing-md);
}

.grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: var(--spacing-md);
}

/* Tablet (768px+) */
@media (min-width: 48rem) {
  .grid {
    grid-template-columns: repeat(2, 1fr);
    gap: var(--spacing-lg);
  }
}

/* Desktop (1024px+) */
@media (min-width: 64rem) {
  .grid {
    grid-template-columns: repeat(3, 1fr);
  }

  .page-layout {
    display: grid;
    grid-template-columns: 280px 1fr;
  }
}

/* Large desktop (1440px+) */
@media (min-width: 90rem) {
  .grid {
    grid-template-columns: repeat(4, 1fr);
  }
}
```

Modern CSS approaches:

```
/* Container queries (modern browsers) — responsive to parent, not viewport */
.card-container {
  container-type: inline-size;
}

@container (min-width: 400px) {
  .card {
    flex-direction: row;
```

```
  }
}

/* clamp() for fluid typography — no media queries needed */
h1 {
  font-size: clamp(1.5rem, 3vw + 1rem, 3rem);
}

/* Responsive images */
img {
  max-width: 100%;
  height: auto;
}
```

What to listen for:

- Uses mobile-first (min-width) breakpoints
- Uses `rem` for breakpoints (not `px`) for accessibility
- Knows modern techniques (container queries, `clamp()`)
- Mentions CSS custom properties for consistency
- Considers fluid typography

Red flag: Uses `max-width` (desktop-first) without a reason. Hard-codes pixel values everywhere. Does not know container queries exist.

D3. CSS Specificity and BEM

Gate 1 | Q41: How does CSS specificity work?

Question: Explain CSS specificity. How do you resolve styling conflicts without using `!important`?

Expected Answer:

Specificity is a scoring system that determines which CSS rule applies when multiple rules target the same element:

Selector type	Specificity weight	Example
Inline styles	1,0,0,0	`style="color: red"`
ID selectors	0,1,0,0	`#header`
Class, attribute, pseudo-class	0,0,1,0	`.nav`, `[type="text"]`, `:hover`
Element, pseudo-element	0,0,0,1	`div`, `::before`
Universal, combinators	0,0,0,0	`*`, `>`, `+`, `~`

```
/* Specificity: 0,0,1,0 (one class) */
.button { color: blue; }
```

```
/* Specificity: 0,0,2,0 (two classes) — wins over above */
.button.primary { color: green; }

/* Specificity: 0,1,0,0 (one ID) — wins over classes */
#submit-button { color: red; }

/* Specificity: 0,0,1,1 (one class + one element) */
div.button { color: purple; }
```

BEM naming convention avoids specificity wars by keeping selectors flat:

```
/* Block */
.card { }

/* Element (belongs to the block) */
.card__header { }
.card__body { }
.card__footer { }

/* Modifier (variation of block or element) */
.card--featured { }
.card__header--large { }
```

```
<div class="card card--featured">
  <div class="card__header card__header--large">Title</div>
  <div class="card__body">Content</div>
  <div class="card__footer">Actions</div>
</div>
```

Strategies to avoid `!important`:

1. Use BEM or another flat naming convention (specificity stays at 0,0,1,0)
2. Use CSS custom properties for theme values
3. Use cascade layers (`@layer`) to control which styles take precedence
4. In Angular: use `ViewEncapsulation` (default Emulated) to scope styles per component

What to listen for:

- Can explain the specificity scoring system
- Knows that ID selectors beat class selectors
- Uses BEM or another methodology to keep specificity low
- Does not recommend `!important` as a solution

D4. Accessibility

Gate 2 | Q42: What are the key accessibility practices for web applications?

Question: How do you ensure an Angular application is accessible? Give specific examples.

Expected Answer:

1. Semantic HTML:

```
<!-- BAD — no semantic meaning -->
<div class="header">
  <div class="nav">
    <div class="link" (click)="navigate()">Home</div>
  </div>
</div>

<!-- GOOD — semantic elements provide meaning to assistive technology -->
<header>
  <nav aria-label="Main navigation">
    <a routerLink="/">Home</a>
  </nav>
</header>
```

2. ARIA attributes (when semantic HTML is not enough):

```
<!-- Custom component needs ARIA -->
<div role="tablist" aria-label="Dashboard sections">
  <button role="tab"
          [attr.aria-selected]="activeTab === 'overview'"
          (click)="selectTab('overview')">
    Overview
  </button>
  <button role="tab"
          [attr.aria-selected]="activeTab === 'details'"
          (click)="selectTab('details')">
    Details
  </button>
</div>
<div role="tabpanel" [attr.aria-labelledby]="activeTab + '-tab'">
  <!-- Tab content -->
</div>

<!-- Loading states -->
<div aria-live="polite" aria-busy="loading">
  <span *ngIf="loading">Loading data...</span>
</div>

<!-- Icon buttons need labels -->
<button aria-label="Close dialog" (click)="close()">
  <mat-icon>close</mat-icon>
</button>
```

3. Keyboard navigation:

```
// Ensure custom components are keyboard-accessible
@Component({
  selector: 'app-dropdown',
  template: `
    <div role="listbox"
         tabindex="0"
         (keydown.arrowDown)="selectNext()"
         (keydown.arrowUp)="selectPrevious()"
         (keydown.enter)="confirmSelection()"
         (keydown.escape)="close()">
      <div *ngFor="let option of options"
           role="option"
           [attr.aria-selected]="option === selected">
        {{ option.label }}
      </div>
    </div>
  `,
})
export class DropdownComponent { /* ... */ }
```

4. Focus management (for SPAs):

```
// After route changes, move focus to main content
@Component({ /* ... */ })
export class AppComponent {
  private router = inject(Router);

  constructor() {
    this.router.events.pipe(
      filter(event => event instanceof NavigationEnd),
    ).subscribe(() => {
      const main = document.querySelector('main');
      if (main) {
        main.focus();
      }
    });
  }
}
```

5. Testing with tools:

- axe DevTools browser extension for automated checks
- Lighthouse accessibility audit
- Screen reader testing (NVDA, VoiceOver)
- `@angular-eslint` rules for template accessibility

What to listen for:

- Starts with semantic HTML before reaching for ARIA
- Knows the "first rule of ARIA": do not use ARIA if native HTML can do it
- Considers keyboard navigation for custom components
- Mentions focus management for SPA route changes
- Uses `aria-live` for dynamic content announcements
- Names specific testing tools

Red flag: Only mentions adding `alt` to images. Does not know about ARIA roles. Uses `<div>` with `(click)` without keyboard support. Cannot name any accessibility testing tool.

↑ Back to Index

Appendix A — Angular CLI Quick Reference

```
# Project creation
ng new <name> [--standalone] [--routing] [--style=scss] [--ssr]
ng new <name> --no-standalone    # module-based (legacy)

# Generate
ng g c <path/name>               # component (standalone by default in v17+)
ng g s <path/name>               # service
ng g m <path/name>               # module
ng g guard <path/name>           # route guard (--functional for function-based)
ng g pipe <path/name>            # pipe
ng g directive <path/name>       # directive
ng g interceptor <path/name>     # HTTP interceptor
ng g resolver <path/name>        # route resolver
ng g enum <path/name>            # enum
ng g interface <path/name>       # interface
ng g class <path/name>           # class
ng g library <name>              # library (workspace)

# Serve
ng serve                         # http://localhost:4200
ng serve -o                      # open browser
ng serve --port 4300             # custom port
ng serve --configuration=staging # named config
ng serve --proxy-config proxy.conf.json  # API proxy

# Build
ng build                         # dev build
ng build --configuration=production
ng build --stats-json            # for bundle analyzer

# Test
ng test                          # Karma (watch mode)
ng test --no-watch               # single run
ng test --code-coverage
ng test --include='**/user.service.spec.ts'  # run specific test
```

```
# Lint
ng lint

# Update
ng update                        # check for updates
ng update @angular/core @angular/cli
ng update @angular/core --force  # skip peer dependency checks

# Environment
ng generate environments         # Angular 15+ environment files

# Add packages
ng add @angular/material
ng add @angular/pwa
ng add @angular-eslint/schematics
```

↑ Back to Index

Appendix B — RxJS Operators Cheat Sheet

Creation Operators

Operator	Description	Example
`of(1, 2, 3)`	Emit values synchronously, then complete	Quick mock data
`from([1, 2, 3])`	Convert array/iterable/Promise to Observable	Wrap Promise
`interval(1000)`	Emit incrementing numbers every N ms	Polling
`timer(2000)`	Emit 0 after N ms, then complete	Delayed action
`fromEvent(el, 'click')`	Create Observable from DOM event	Event handling
`EMPTY`	Emit nothing, complete immediately	catchError fallback
`throwError(() => err)`	Emit error immediately	Error testing
`defer(() => obs$)`	Defer Observable creation until subscription	Lazy evaluation

Transformation Operators

Operator	Description	When to use
`map(x => x * 2)`	Transform each emission	Data mapping
`switchMap(x => obs$)`	Cancel previous inner, subscribe to new	Typeahead, route params
`mergeMap(x => obs$)`	Run all inner Observables concurrently	Parallel fire-and-forget

Operator	Description	When to use
`concatMap(x => obs$)`	Queue inner Observables sequentially	Ordered operations
`exhaustMap(x => obs$)`	Ignore new while inner is active	Submit button guard
`scan((acc, val) => ..., seed)`	Reduce over time (like running total)	Accumulating state
`pairwise()`	Emit previous and current as pair	Comparing changes
`pluck('key')`	Extract property value (deprecated, use `map`)	

Filtering Operators

Operator	Description	When to use
`filter(x => x > 0)`	Only pass values matching predicate	Conditional logic
`distinctUntilChanged()`	Skip consecutive duplicates	Prevent re-renders
`debounceTime(300)`	Wait N ms after last emission	Search input
`throttleTime(1000)`	Emit first, then ignore for N ms	Scroll events
`take(5)`	Take first N values, then complete	Limited results
`takeUntil(notifier$)`	Take until notifier emits	Unsubscribe on destroy
`skip(1)`	Skip first N emissions	Skip initial value
`first()`	Take first value, then complete	One-shot
`last()`	Wait for complete, emit last value	Final value

Combination Operators

Operator	Description	When to use
`combineLatest([a$, b$])`	Emit when any source emits (after all have emitted once)	Multiple input dependencies
`forkJoin([a$, b$])`	Wait for all to complete, emit last values	Parallel HTTP calls
`merge(a$, b$)`	Merge multiple streams into one	Event aggregation
`zip(a$, b$)`	Pair emissions by index	Synchronized streams
`withLatestFrom(other$)`	Combine with latest from another source	Action + current state
`startWith(value)`	Emit initial value before source	Default state
`concat(a$, b$)`	Subscribe to next after previous completes	Sequential streams

Error Handling Operators

Operator	Description	When to use
`catchError(err => obs$)`	Handle error, return replacement Observable	Error recovery
`retry(3)`	Resubscribe on error, up to N times	Transient failures
`retry({ count: 3, delay: 1000 })`	Retry with delay	Backoff
`retryWhen(notifier => ...)`	Custom retry logic (deprecated in v7, use retry config)	
`finalize(() => ...)`	Run on complete or error (like `finally`)	Cleanup

Utility Operators

Operator	Description	When to use
`tap(x => console.log(x))`	Side effect without modifying stream	Logging/debugging
`delay(1000)`	Delay each emission by N ms	Testing, animation
`shareReplay(1)`	Multicast + replay last N values	Shared HTTP cache
`share()`	Multicast without replay	Hot observables

↑ Back to Index

Gate Assessment Summary

Use this rubric to evaluate candidates at each gate. Mark each area as Pass / Partial / Fail.

Gate 1 — Junior (0-3 years)

#	Area	Must demonstrate
1	JS Closures & Scope	Explain closures, hoisting, TDZ
2	JS Hoisting	Predict var/let behavior correctly
3	ES6+ Basics	Destructuring, spread, rest syntax
4	TS Basics	interface vs type, when to use each
5	TS Type Safety	any vs unknown vs never
6	Angular Components	NgModules vs standalone
7	Angular Lifecycle	Name hooks in order, explain top 3
8	Angular CLI	Generate commands, serve, build
9	Promises/async	Convert callbacks, error handling

#	Area	Must demonstrate
10	HTML/CSS	Flexbox vs Grid, specificity

Pass criteria: 7/10 areas at Pass level. Can explain concepts clearly and predict code output correctly.

Gate 2 — Mid (3-7 years)

#	Area	Must demonstrate
1	Closure pitfalls	Solve the loop problem, explain fix
2	Event loop	Predict microtask/macrotask order
3	Prototype chain	Explain inheritance under the hood
4	`this` binding	Predict output for all 4 cases
5	Event delegation	Implement efficiently for large lists
6	TS Generics	Build generic types, use utility types
7	Angular DI	providedIn, hierarchical injectors
8	Angular Routing	Lazy loading, functional guards
9	RxJS Operators	switchMap vs mergeMap vs concatMap
10	Angular Forms	Reactive forms, custom validators
11	Angular HTTP	Interceptors, auth token handling
12	Angular Testing	HttpTestingController, service tests
13	Signals	Explain signals vs observables
14	Responsive CSS	Mobile-first, modern techniques
15	Accessibility	ARIA, keyboard nav, semantic HTML

Pass criteria: 10/15 areas at Pass level. Can explain tradeoffs, not just definitions. Gives real-world examples.

Gate 3 — Senior (7-15 years)

#	Area	Must demonstrate
1	Event loop debugging	Diagnose frozen UI scenario
2	Async patterns	Promise.all vs allSettled, parallel execution
3	TS Type Guards	Discriminated unions, exhaustiveness
4	Decorators	Explain runtime behavior of Angular decorators
5	RxJS memory leaks	Diagnose and prevent subscription leaks
6	Change detection	OnPush optimization, Signals migration

#	Area	Must demonstrate
7	Route architecture	Resolver vs guard vs component init tradeoffs
8	Error handling	Layered HTTP error strategy
9	State management	Choose appropriate tool by state complexity
10	Bundle optimization	Analyze and reduce bundle size
11	Component testing	Mock ActivatedRoute, complex dependencies
12	Component communication	Alternatives to prop drilling
13	Security	XSS, CSRF, secure storage

Pass criteria: 8/13 areas at Pass level. Proposes solutions with tradeoffs. Can lead a team through architectural decisions. Considers edge cases without prompting.

Gate 4 — Lead/Architect (15+ years)

#	Area	Must demonstrate
1	Reactive state design	Build service-based state with undo, concurrency
2	Micro-frontend architecture	Module Federation, shared deps, cross-MFE comms
3	Real-time collaboration	WebSocket + OT/CRDT + optimistic updates
4	System design breadth	Consider deployment, observability, team workflows

Pass criteria: 3/4 areas at Pass level. Designs for team scalability, not just technical correctness. Considers organizational and operational concerns alongside code.

Total questions in this handbook: 42

Part	Topic	Questions
A	JavaScript Fundamentals	Q1 - Q11 (11 questions)
B	TypeScript	Q12 - Q16 (5 questions)
C	Angular Deep Dive	Q17 - Q38 (22 questions)
D	HTML/CSS Essentials	Q39 - Q42 (4 questions)

↑ Back to Index